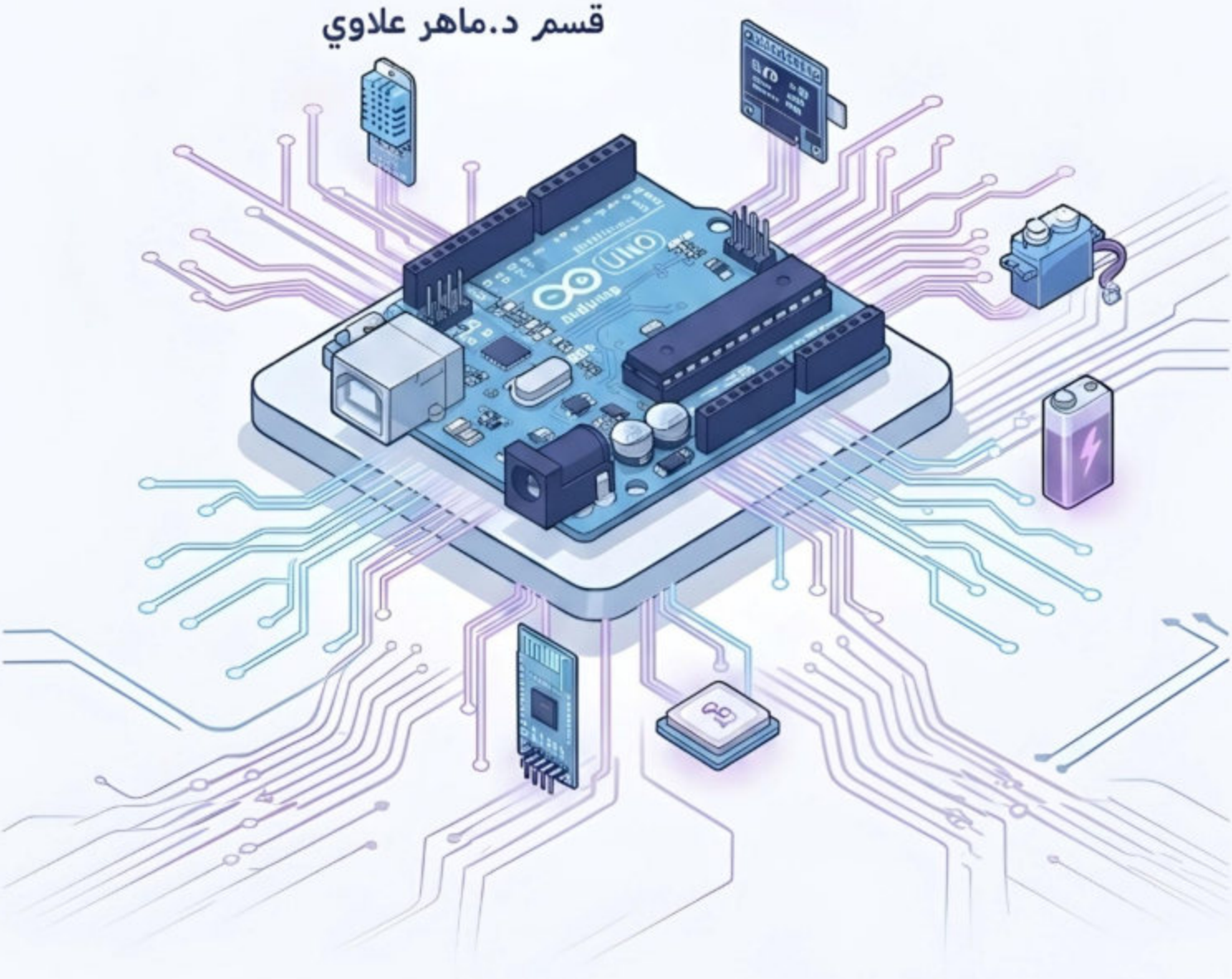
فريق نون التطوعي - السنة الرابعة

المعالجات الصغرية و نظمها

الملحق الثاني :

تطبيقات الآردوينو

قسم د.ماهر علاوي



كتابة وتدقيق: بلال السابق

تفريغ: سنا ذوالغنى - ليلاس إدلبي

قسم هندسة الالكترونيات والاتصالات

2025 - 2026

لوحات الآردوينو

ماهو الأردوينو ؟

- ✓ لوحة الكترونية صغيرة الحجم ، مفتوحة المصدر ، رخيصة الثمن ، سهلة الاستخدام ، تستخدم في المشاريع الالكترونية المختلفة .
- ✓ تم تطوريها في إيطاليا عام 2005 لتسهيل تعليم الطلاب على استخدام microcontroller.
- ✓ تعمل كمتحكم ورابط بين المكونات الكهربائية والالكترونية المختلفة .
- ✓ يمكن ربط مداخل معها مثل : لوحة أزرار ، مستقبل أشعة تحت الحمراء ، بلوتوث ، حساسات ....
- ✓ يمكن ربط مفعلات Actuators على خرجها مثل : ريليه ، محركات ، شاشات ، LED ....

مزايا لوحات الأردوينو :



- ❖ سهلة الاستخدام .
- ❖ منخفضة السعر .
- ❖ لها إضافات كثيرة متوافقة (shields) .
- ❖ توفر موقع منظم و مفيد جداً وهو Arduino.cc .
- ❖ الشهرة الواسعة و توفر الكثير من الشروحات على الانترنت .
- ❖ تستخدم لغة برمجة عالية المستوى وهي Arduino C باستخدام بيئة Arduino ide البسيطة .
- ❖ إمكانية اتصالها بالانترنت عبر WiFi .

عيوبها :

- ○ القدرة البرمجية أقل بكثير من الحاسب .
- ○ غير قادرة على تشغيل أنظمة مثل windows أو Android أو شاشات ذات وضوع عالٍ .



✧ مثل هذه الاستخدامات تتطلب حاسب أو لوحات متقدمة مثل Raspberry Pi

بعض أشهر التعليمات الشائعة في Arduino C :

الشرح	التعليمة
تضمين مكتبة المحرك الخطوي	#include <Stepper.h>
التصريح عن الاعدادات العامة	void setup() {….
الكتلة البرمجية المنفذة بشكل مستمر	void loop() {….
تهيئة (فتح) المنفذ التسلسلي بسرعة 9600 kbps	Serial.begin(9600);
تحديد سرعة المحرك الخطوي	stepper.setSpeed(200);
إرسال قيمة عبر المنفذ التسلسلي	Serial.print(Pval)
إرسال قيمة عبر المنفذ التسلسلي والانتقال لسطر جديد	Serial.println(Pval)
تصريح عن متغير صحيح قيمته صفر	int val = 0;
تعيين المنفذ المسمى led كمخرج	pinMode(led, OUTPUT);
تعيين المنفذ المسمى pin كمدخل	pinMode(pin, INTPUT);
إخراج قيمة تماثلية على المخرج led	analogWrite(led, brightness);
إخراج قيمة رقمية (واحد منطقي)على المخرج led	digitalWrite(led, HIGH);
قراءة قيمة تماثلية على المدخل pin	X= analogRead(pin);
تأخير زمني بمقدار 500 ميكروثانية	delayMicroseconds(500);
تأخير زمني بمقدار 1000 ميلي ثانية أي ثانية واحدة	delay (1000);
التعريف عن قيمة حقيقية	float val;
قيمة جيب الزاوية مع تحويل للراديان	v = sin(3.14 * x / 180);
حلقة for	for (int x=0; x<5; x++) {…
التحقق من شرط مساواة	if (x==5) {….
التحقق من شرطين معاً (.)	if (x==5 && y==10) {….
التحقق من أحد الشروط (+)	if (x==5 || y==10) {…..
خلط بين الحالتين السابقتين	if (x==5 && (y==10 || z==25)) {…
التحقق من نفي قيمة ما	if (!x) {…….
أخذ القيمة الصحيحة من قيمة حقيقية	ledVal = int(sin(Val)*255)));

المكونات الأساسية للوحة Arduino UNO :

1] متحكم Atmega328p بالمواصفات التي درسناها سابقاً

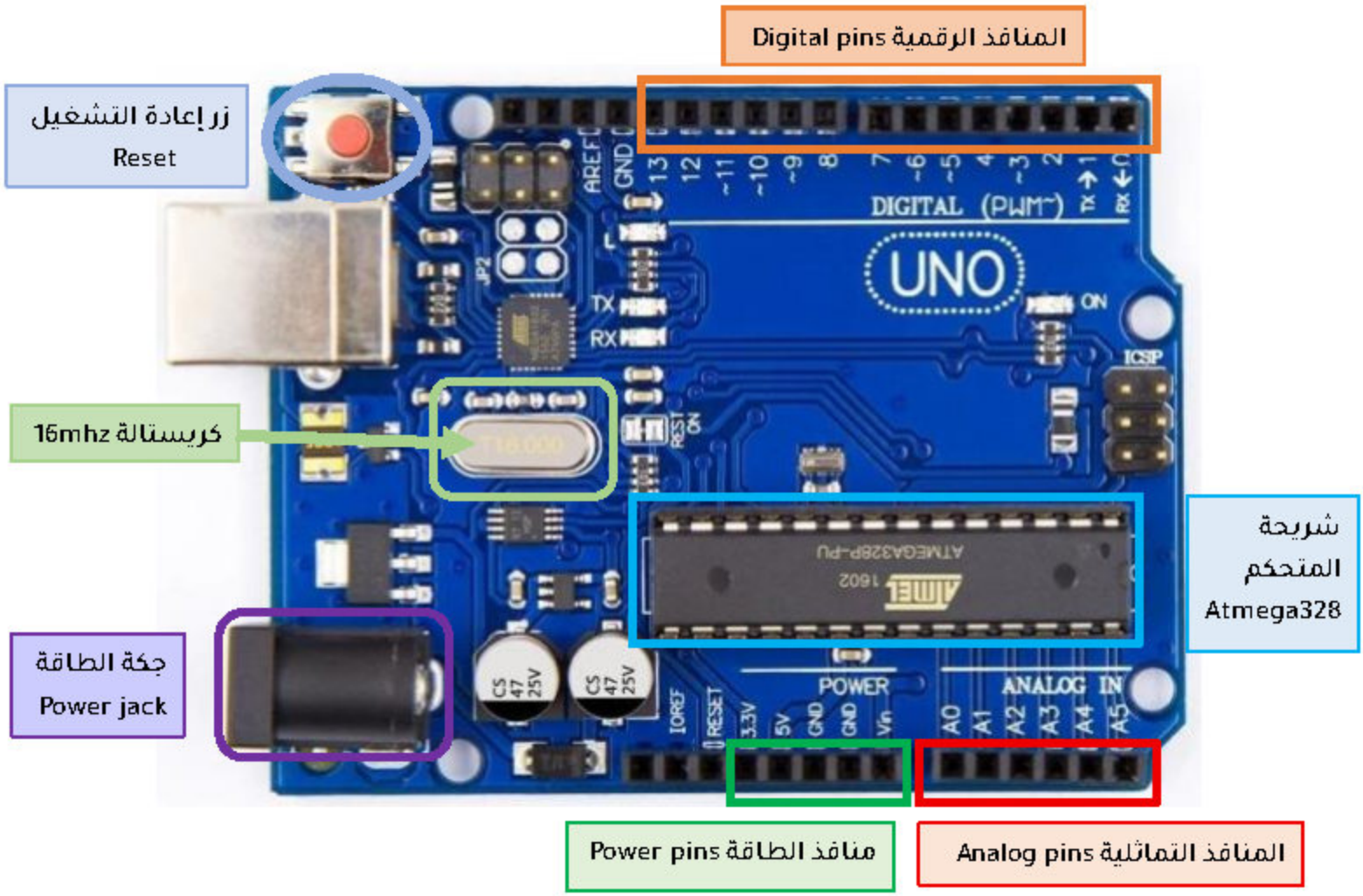
2] منافذ رقمية عددها 14 منفذ مرقمة من 0 حتى 13 و يمكنها العمل كمداخل أو مخارج بحسب الكود

المنفذ الرقمي يمكن أن يمد الحمل بجهد 5v وتيار يصل إلى 20mA ، وهذا التيار كافٍ لتشغيل LED لكنه غير قادر على تشغيل ريليه مثلاً لذلك نستخدم ترانزستور لقيادة الريليه .

العلامة ~ تعني أن الطرف يصلح لاخراج قيمة تماثلية ويستخدم تعديل عرض النبضة PWM

3] المنافذ التماثلية عددها 6 منافذ من A0 حتى A5 يمكنها قياس الجهد تماثلياً

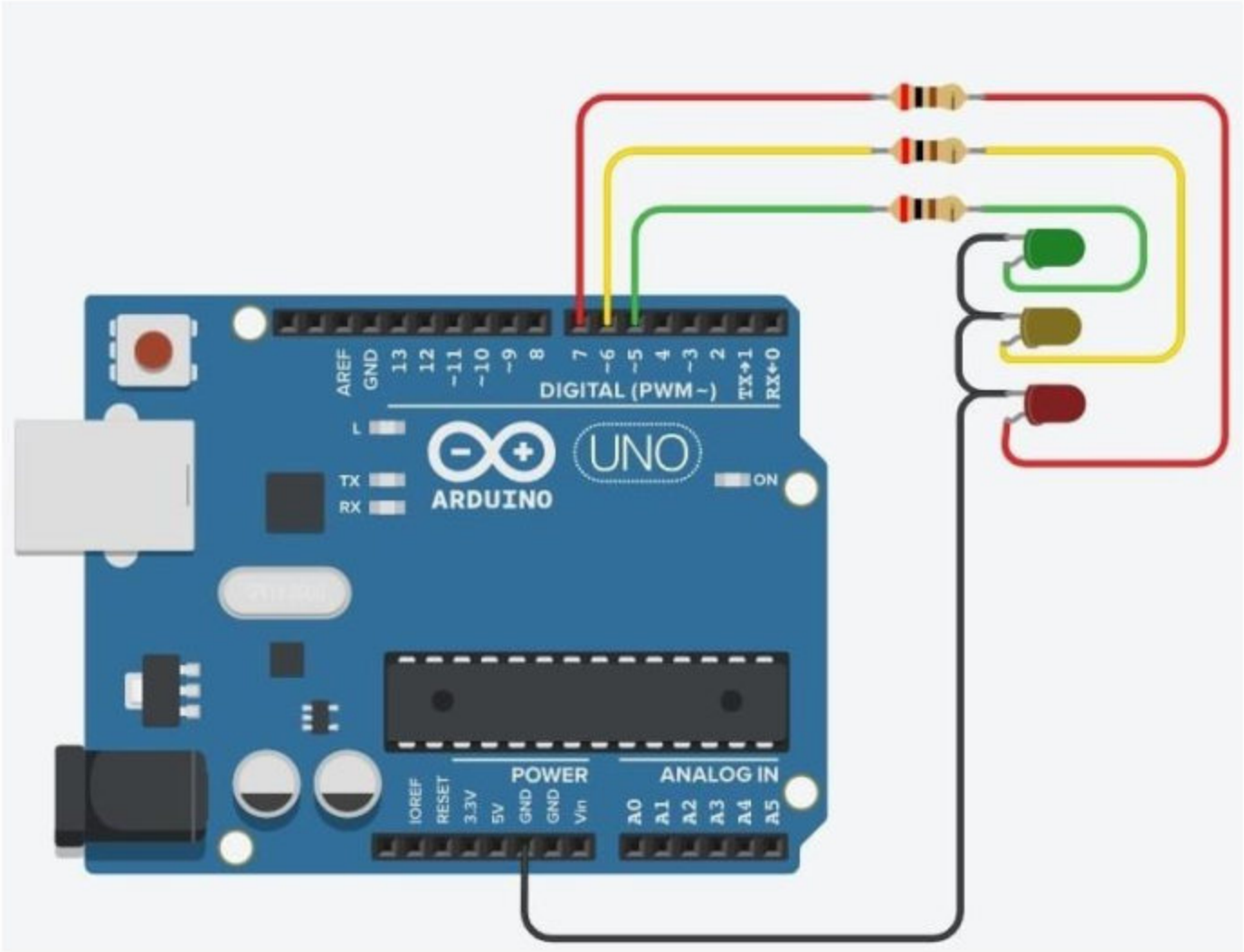
4] منافذ الطاقة : يوجد مخرج 5v ومخرج 3.3v وعدة مخارج GND ، ويمكن تغذية الأردوينو عبر المنفذ V_{in} من خلال تغذيته بجهد 7-12 v ، أو عن طريق Power Jack .



تطبيقات الآردوينو

التطبيق الأول : إشارارت المرور:

يقوم هذا التطبيق بمحاكاة عمل إشارات المرور تقريباً ، حيث يتم إضاءة الثنائي الضوئي الأحمر لمدة 5 ثوانٍ ، يتبعه الثنائي الضوئي الأصفر لمدة 2 ثانية ، ثم اطفاؤهما وتشغيل الثنائي الضوئي الأخضر لمدة 5 ثوانٍ و من ثم العودة إلى الأصفر لمدة 2 ثانية و هكذا....



الكود:

```
int ledDelay = 5000;                مدة التأخير الأساسية 5 ثواني //
int redPin =7;                      التصريح عن المتغير redPin لتخزين فيه رقم النقذ الموصول عليه الضوء الأحمر //
int yellowPin = 6;                  التصريح عن المتغير yellowPin لتخزين فيه رقم النقذ الموصول عليه الضوء الأصفر //
int greenPin = 5;                   التصريح عن المتغير greenPin لتخزين فيه رقم النقذ الموصول عليه الضوء الأخضر //

void setup() {
  pinMode (redPin, OUTPUT);         // (Output) جعل النقذ 7 خرج
  pinMode (yellowPin, OUTPUT);      // (Output) جعل النقذ 6 خرج
  pinMode (greenPin, OUTPUT);       // (Output) جعل النقذ 5 خرج
}

void loop() {

  digitalWrite(redPin, HIGH);       // (HIGH = 5V إعطاء) تشغيل LEDالأحمر
  delay(ledDelay);                  // انتظار 5 ثواني مع بقاء الأحمر شغال

  digitalWrite(yellowPin, HIGH);    // (HIGH = 5V إعطاء) تشغيل LED الأصفر
  delay(2000);                      // انتظار 2 ثانية مع بقاء الأصفر شغال

  digitalWrite(greenPin, HIGH);     // (HIGH = 5V إعطاء) تشغيل LEDالأخضر
  digitalWrite(redPin, LOW);        // (LOW = 0V إعطاء) ايقاف LEDالأحمر
  digitalWrite(yellowPin, LOW);     // (LOW = 0V إعطاء) ايقاف LEDالأصفر

  delay(ledDelay);                  // استمرار الأخضر لمدة 5 ثواني

  digitalWrite(yellowPin, HIGH);    // تشغيل LED الأصفر
  digitalWrite(greenPin, LOW);      // ايقاف الأخضر //

  delay(2000);                      // انتظار 2 ثانية

  digitalWrite(yellowPin, LOW);     // ايقاف الأصفر //

}                                   // يعيد تنفيذ نفس التسلسل بشكل مستمر loop //
```

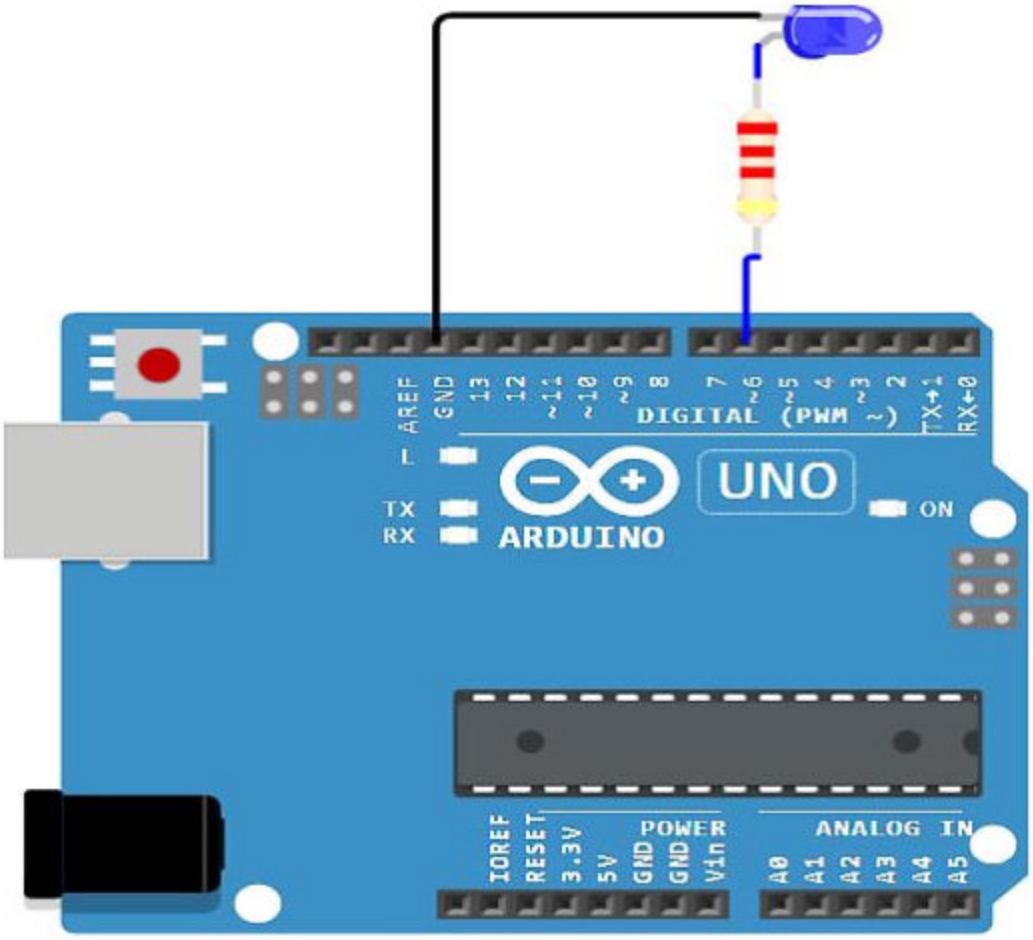


يمكنك مشاهدة فيديو لمحاكاة الدارة السابقة على قناتنا على يوتيوب

إما بالضغط على الرابط التالي: محاكاة إشارة المرور من هنا

أو بمسح الـ QR جانباً :

التطبيق الثاني : إخراج قيم تماثلية على مخارج المتحكم



يعمل التطبيق الثاني على إضاءة LED المرتبط بالمنفذ بشكل متدرج من ثم إطفائه بشكل متدرج وهكذا...
وفي هذه الحالة يجب استخدام أحد المخارج التماثلية PWM الستة المتوفرة في اللوحة وهنا تم استخدام المنفذ I/O6

يمكنك مشاهدة فيديو محاكاة لعمل هذه الدارة من هنا

أو عبر مسح هذا الـ QR

```
int led = 6;                          // تعريف المتغير led وتخزين به رقم المنفذ 6
int brightness = 0;                   // متغير لتخزين قيمة سطوع الـ LED ويبدأ من 0 (مطفأ)
int fadeAmount = 5;                   // مقدار الزيادة أو النقصان في السطوع بكل دورة

void setup( ) {
  pinMode(led, OUTPUT);               // ضبط المنفذ 6 كمخرج لإرسال إشارة إلى الـ LED
}

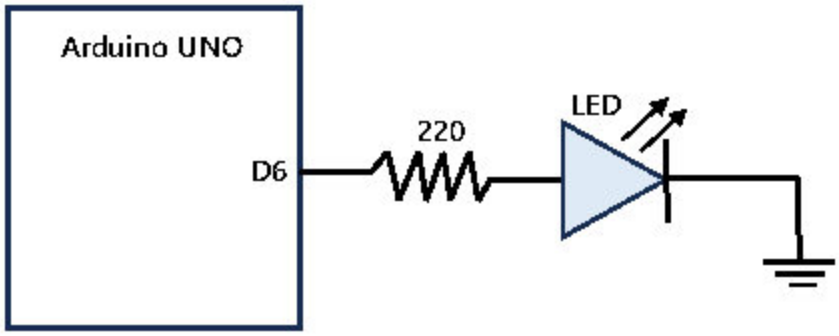
void loop( ) {

  analogWrite(led, brightness);       // إرسال قيمة PWM إلى LED للتحكم بالسطوع حسب قيمة brightness
  brightness = brightness + fadeAmount;   // زيادة أو إنقاص قيمة السطوع بمقدار fadeAmount

  if (brightness <= 0 || brightness >= 255) {   // التحقق إذا وصل السطوع إلى الحد الأدنى 0 أو الأعلى 255
    fadeAmount = -fadeAmount; }         // عكس الإشارة: إذا كان يزيد يصبح ينقص وإذا كان ينقص يصبح يزيد
  delay(30);                          // انتظار 30 ميلي ثانية لإبطاء تغير السطوع وجعل التلاشي مرئياً
}
```

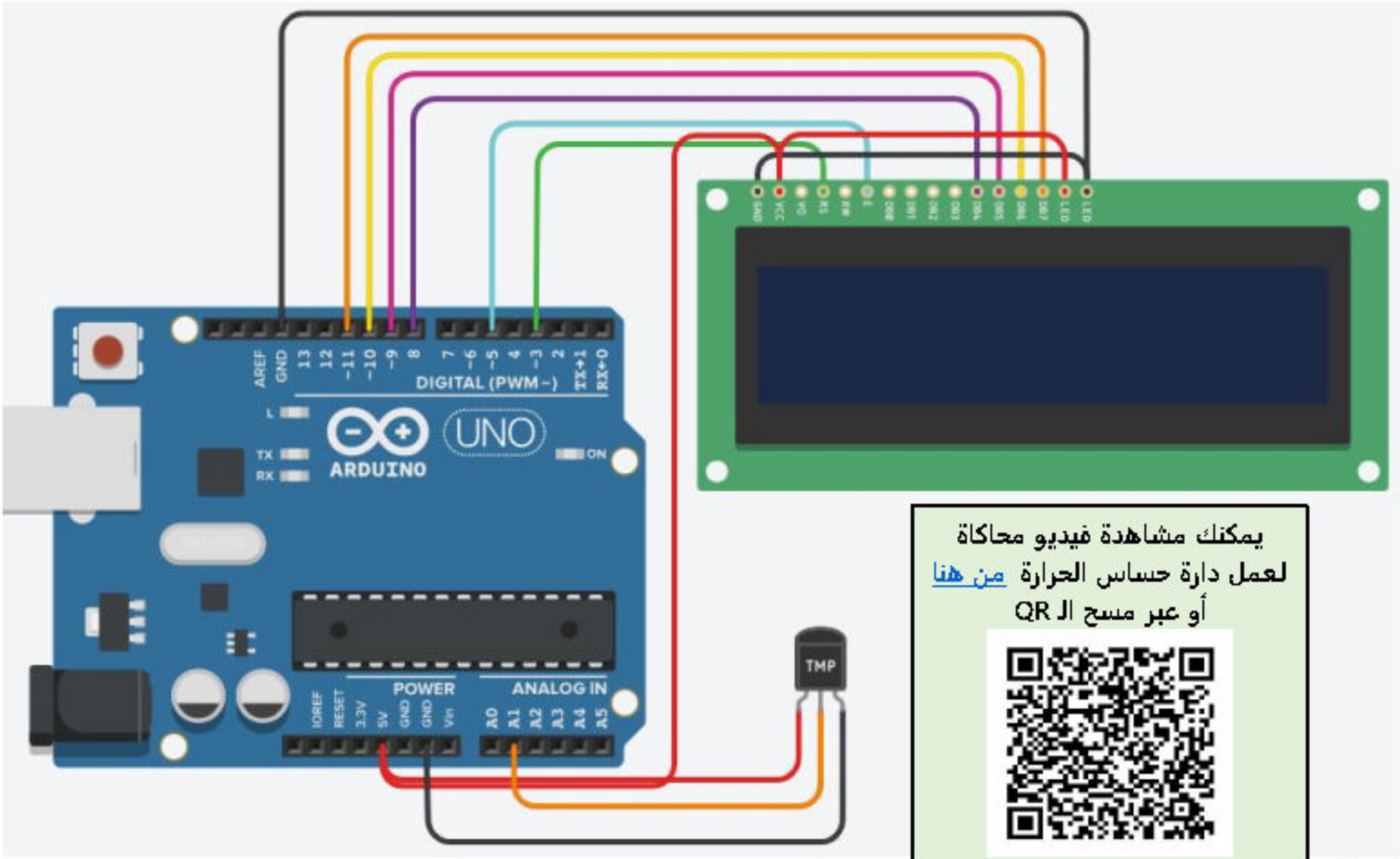
ملاحظة : الرسم مطلوب بالامتحان ، بالتأكيد لن نرسم اللوحة بتفاصيلها وإنما رسم تقريبي بسيط لتوضيح عمل الدارة وتوصيلاتها فقط ، لذا سيتم توضيح الرسم الامتحاني المطلوب لكل تطبيق بما يضمن العلامة الكاملة بإذن الله .

Fade circuit :



التطبيق الثالث : حساس الحرارة

يقوم التطبيق التالي بقراءة الجهد التماثلي الناتج عن حساس الحرارة LM35 والمتصل إلى المنفذ التماثلي A1 ، وإظهار الحرارة بشكل رقمي في واجهة الربط التسلسلي مع الحاسب Serial monitor وعلى شاشة LCD 16×2



يمكنك مشاهدة فيديو محاكاة
لعمل دارة حساس الحرارة من هنا
أو عبر مسح الـ QR

```
#include <LiquidCrystal.h>          // تضمين مكتبة التحكم بشاشة LCD
const int sensorPin = A1;           // تعريف الطرف A1 كمدخل لحساس الحرارة
int reading;                        // متغير لتخزين القيمة الرقمية المقروءة من الحساس
float voltage;                      // متغير لتخزين قيمة الجهد المحسوبة من القراءة
float temperatureC;                 // متغير لتخزين درجة الحرارة بالسلسيوس

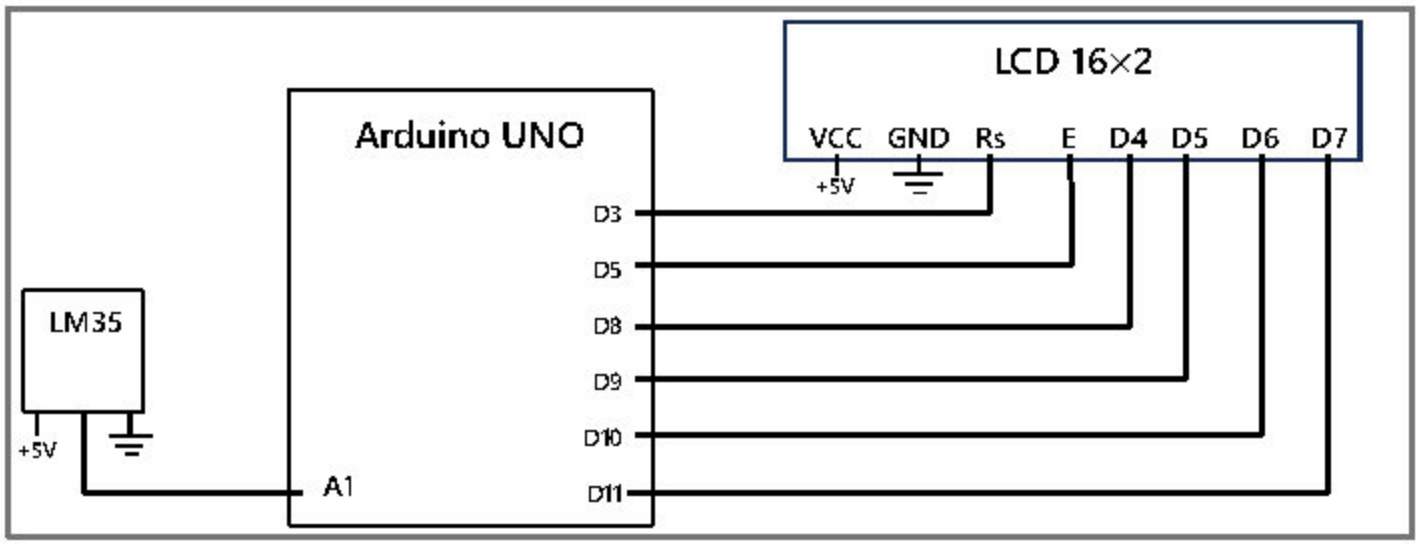
LiquidCrystal lcd(3, 5, 8, 9, 10, 11);   // إنشاء كائن للشاشة مع تحديد أطراف التوصيل مع الأردوينو
                // (rs,en,d4,d5,d6,d7)

void setup() {
  lcd.begin(16, 2);                 // تهيئة شاشة LCD بـ 16 عمود و2 صف
  Serial.begin(9600);               // بدء الاتصال التسلسلي بسرعة 9600 بت في الثانية
}

void loop() {
  reading = analogRead(sensorPin);          // قراءة القيمة التناظرية القادمة من حساس الحرارة
  voltage = reading * 5.0 / 1024;           // تحويل القراءة الرقمية إلى قيمة جهد بالفولت
  temperatureC = (voltage - 0.05) * 100;    // تحويل الجهد إلى درجة حرارة حسب معادلة الحساس
  lcd.print("Temp is: ");                   // عرض النص Temp is على شاشة LCD
  lcd.print(temperatureC);                  // عرض قيمة درجة الحرارة على الشاشة
  Serial.println(temperatureC);             // إرسال قيمة الحرارة إلى Serial Monitor
  delay(500);                               // انتظار نصف ثانية لإعطاء وقت لعرض القيمة
  lcd.clear();                              // مسح جميع البيانات المعروضة على شاشة LCD
  delay(1000);                              // انتظار ثانية واحدة قبل إعادة التكرار
}
```

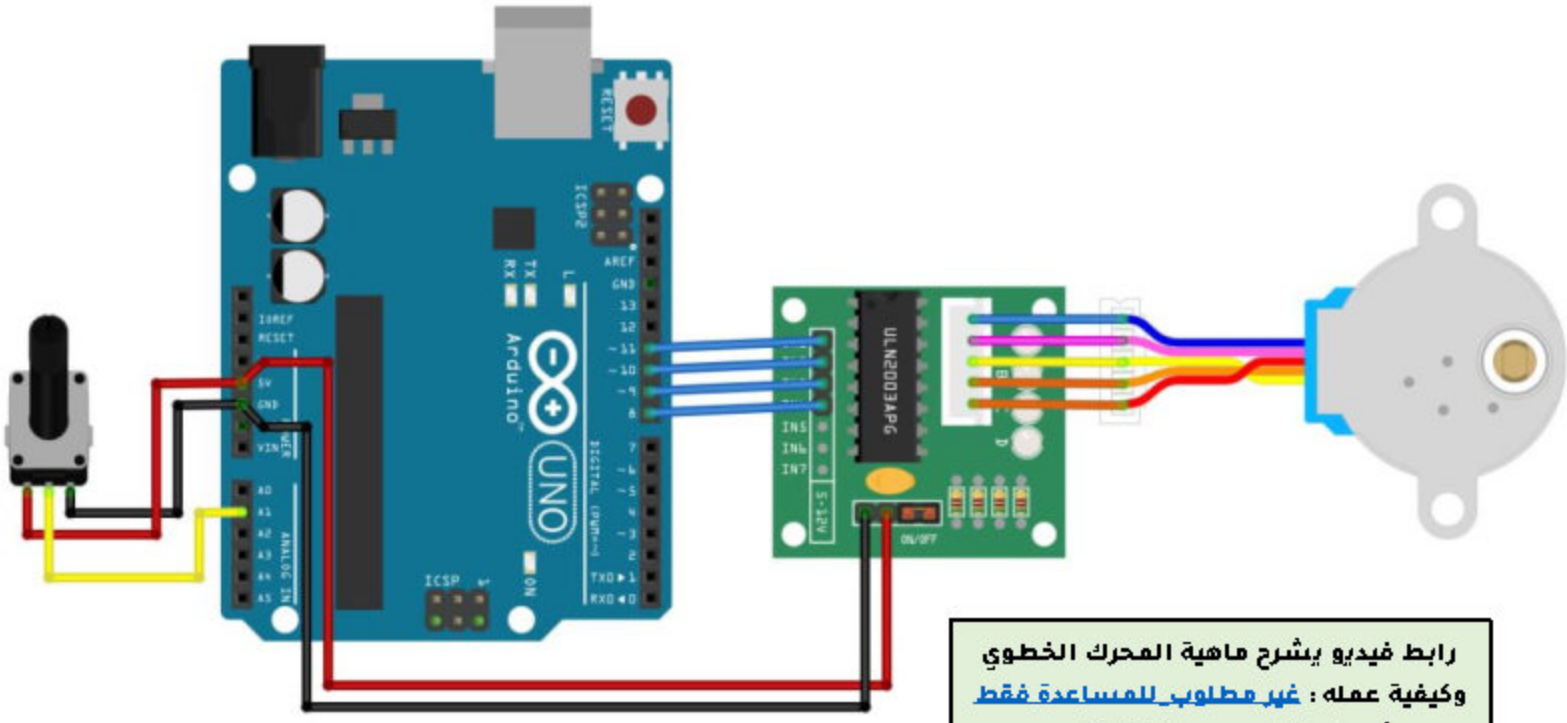
ملاحظات :

- **معادلة تحويل القراءة الرقمية إلى جهد الفولت للحفظ ، وهي معادلة ثابتة نستخدمها دائماً عند تحويل القراءة إلى جهد سواء بهذا التطبيق أو غيره .**
- **في معادلة تحويل القراءة إلى جهد يجب الانتباه إلى الفاصلة في 5.0 لأنه بدون وضع "0." لن يتم التحويل بشكل صحيح.**
- **معادلة تحويل الجهد إلى درجة حرارة للحفظ ،وهي ثابتة من Datasheet الخاصة بالحساس LM35 .**
- **واجهة الربط التسلسلي Serial monitor في برنامج Arduino IDE هي نافذة تستخدم لعرض وإرسال بيانات بين الحاسوب ولوحة الاردوينو وذلك للتجربة أثناء عمل البرنامج .**



التطبيق الرابع : التحكم بالمحرك الخطوي

في هذا التطبيق يتم التحكم بحركة وجهة دوران المحرك الخطوي عن طريق استخدام مقاومة متغيرة ، فعند تدوير المقاومة بجهة معينة يدور المحرك باتجاه معين و عند تدوير المقاومة بالاتجاه الآخر يدور المحرك بالاتجاه المعاكس لجهة الدوران السابقة .



رابط فيديو يشرح ماهية المحرك الخطوي وكيفية عمله : غير مطلوب للمساعدة فقط

أو من خلال مسح الQR التالي:

```
#include <Stepper.h>              // تضمين مكتبة Stepper الخاصة بالتحكم بالمحركات الخطوية
#define STEPS 100                 // تعريف عدد الخطوات في الدورة الواحدة للمحرك

Stepper stepper(STEPS, 8, 10, 9, 11);          // إنشاء كائن للمحرك وربطه بالأرجل 8 و 9 و10 و11

int Pval = 0;                     // متغير لتخزين قيمة المقاومة المتغيرة السابقة
int potVal = 0;                   // متغير لتخزين القيمة الحالية بعد القراءة والمعالجة

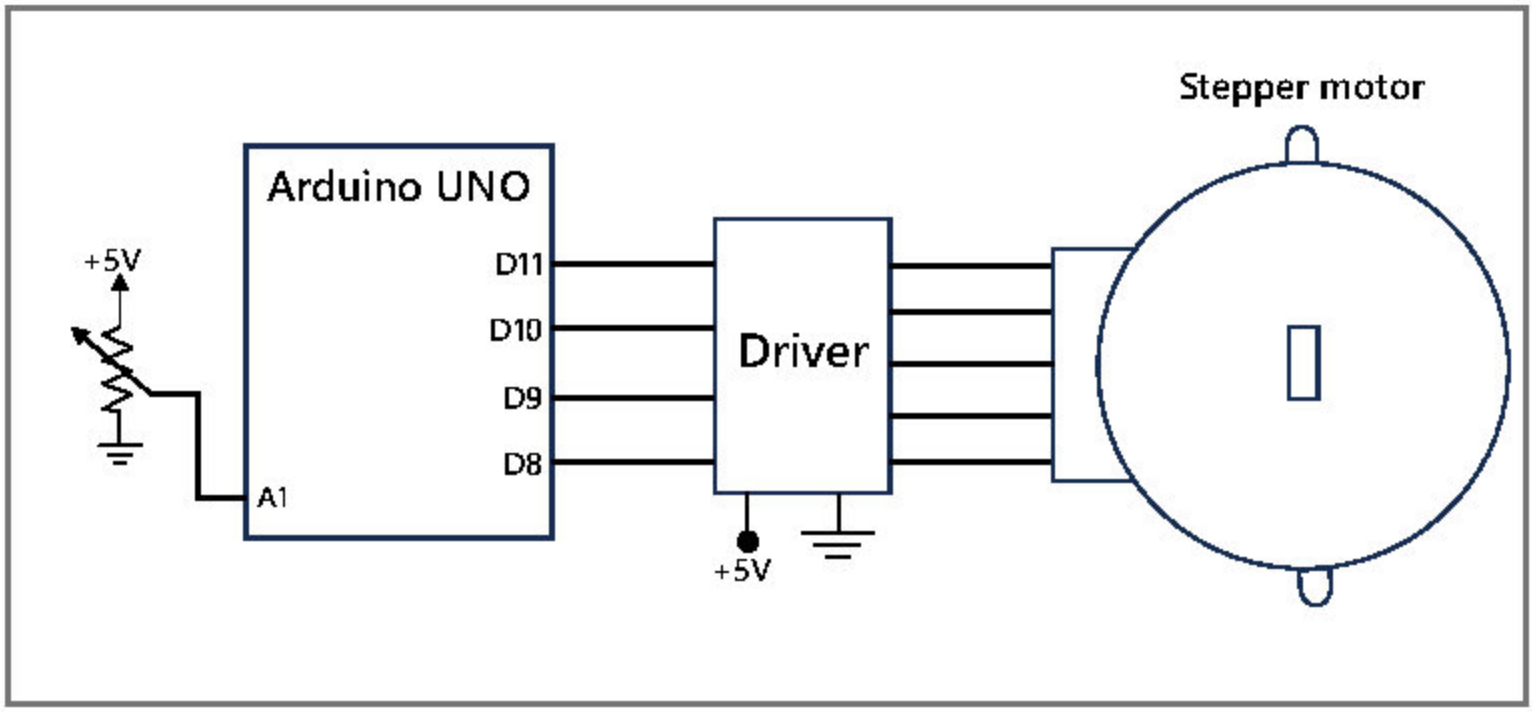
void setup() {
 Serial.begin(9600);              // بدء الاتصال التسلسلي بسرعة 9600 لإرسال القيم إلى Serial Monitor
 stepper.setSpeed(200);           // تحديد سرعة دوران المحرك بوحدة RPM (دورة في الدقيقة)
}
void loop() {
 potVal = map(analogRead(A1), 0, 1024, 0, 500);          // قراءة الجهد من المقاومة المتغيرة وتحويله من 0-1024 إلى 0-500

 if (potVal > Pval)                                      // إذا أصبحت القيمة الحالية أكبر من السابقة فهذا يعني تدوير المقبض باتجاه معين
   stepper.step(5);                                      // تحريك المحرك 5 خطوات باتجاه عقارب الساعة

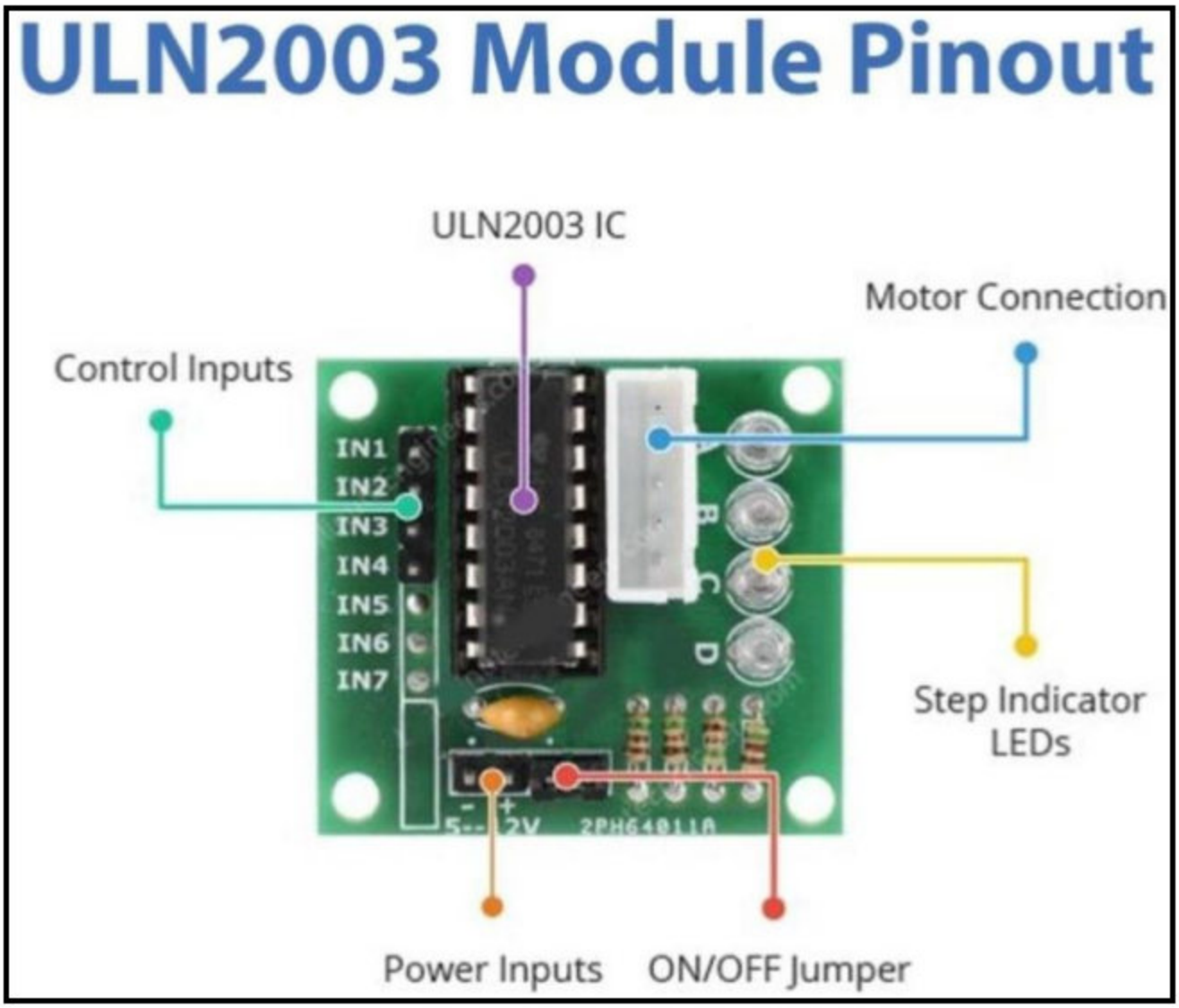
 if (potVal < Pval)                                      // إذا أصبحت القيمة الحالية أصغر من السابقة فهذا يعني تدوير المقبض بالعكس
   stepper.step(-5);                                     // تحريك المحرك 5 خطوات بعكس عقارب الساعة

 Pval = potVal;                                          // حفظ القيمة الحالية لتصبح القيمة السابقة في الدورة القادمة

 Serial.println(Pval);                                   // إرسال القيمة الحالية إلى Serial Monitor لعرضها ومراقبتها
}
```



وكما نلاحظ في الشكل السابق ... لا يتم وصل المحرك الخطوي مباشرة بالأردوينو ؛ لأن المحرك يحتاج تياراً أكبر من قدرة مخارج الأردوينو ، لذلك يُستخدم الDriver ULN2003 لتكبير التيار اللازم لتشغيل ملفات المحرك .

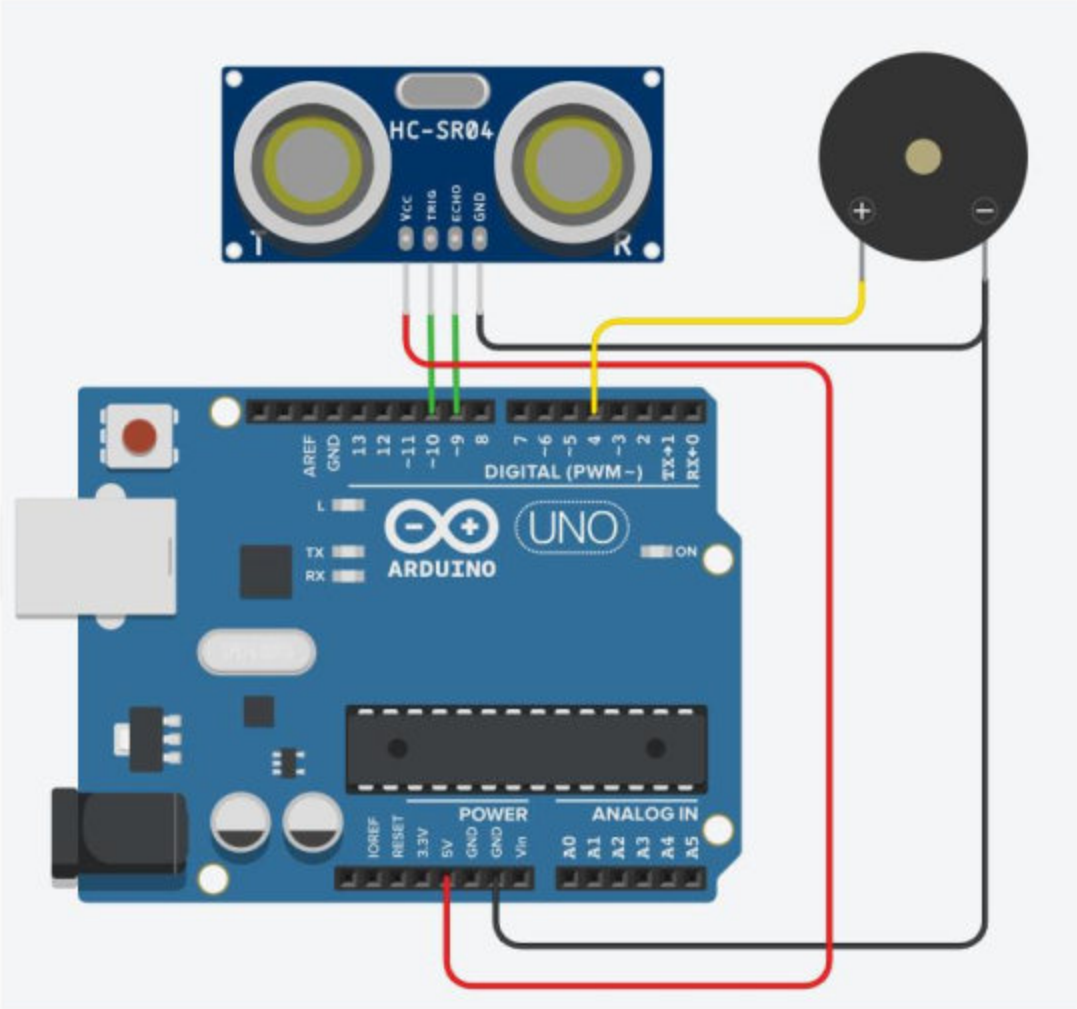


ملاحظة :

الدالة map تستخدم لتحويل قيمة من مجال إلى آخر ، مثال :



التطبيق الخامس : حساس المسافة Ultrasonic



حساس الـ Ultrasonic : هو حساس يُستخدم لقياس المسافة وذلك عن طريق إرسال موجة فوق الصوتية و اصطدامها بالجسم المقابل و عودتها إلى الحساس مرة أخرى، فمن خلال زمن ورود الموجة و ارتدادها نستطيع حساب المسافة !.

يمكنك مشاهدة فيديو توضيحي يشرح آلية عمل هذا الحساس :
من هنا
أو عبر مسح الـ QR :

```
int trigger = 10;          // رجل Trigger الخاصة بحساس HC-SR04 لإرسال النبضة فوق الصوتية
int echo = 9;              // رجل Echo الخاصة باستقبال النبضة المرتدة من الجسم
int buzzer = 4;            // رجل الخرج المستخدم لإطلاق الإنذار
long time = 0;             // متغير لتخزين زمن رحلة الموجة الصوتية ذهاباً وإياباً
long dist = 0;             // متغير لتخزين المسافة المحسوبة بالسنتيمتر

void setup( )
{
  Serial.begin(9600);              // تهيئة الاتصال التسلسلي بسرعة 9600 بت/ثانية لعرض القراءات على Serial Monitor
  pinMode(trigger, OUTPUT);        // ضبط رجل Trigger كمخرج لإرسال نبضة القدح للحساس
  pinMode(echo, INPUT);            // ضبط رجل Echo كمدخل لاستقبال النبضة المرتدة
  pinMode(buzzer, OUTPUT);         // ضبط رجل الخرج كمخرج للتحكم بتشغيله وإيقافه
}

void loop( )
{
  digitalWrite(trigger, LOW);      // التأكد من أن رجل Trigger في الحالة المنخفضة قبل بدء القياس
```

```
  digitalWrite(trigger, HIGH);                    // إرسال نبضة التدخ إلى الحساس
  delay(10);                                      // انتظار 10 ميلي ثانية
  digitalWrite(trigger, LOW);                     // إنهاء نبضة التدخ ليبدأ الحساس عملية القياس

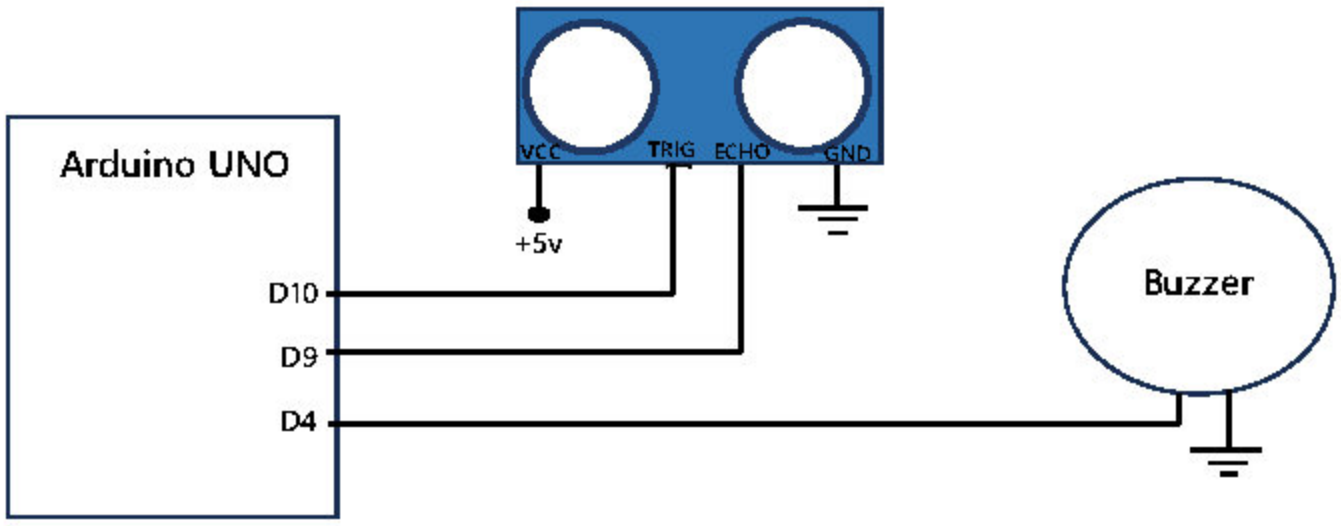
  time = pulseIn(echo, HIGH);                     // قياس الزمن الذي تبقى فيه إشارة Echo مرتفعة والذي يمثل زمن الرحلة الكلي للموجة

  dist = 0.034 * (time / 2);                      // حساب المسافة بالسانتيمتر باستخدام سرعة الصوت 0.034 وتقسيم الزمن على 2 لأن الموجة ذهبت وعادت

  Serial.println(dist);                           // عرض قيمة المسافة على نافذة Serial Monitor لمراقبتها

  if (dist <= 30)                                 // التحقق مما إذا كان الجسم على بعد 30 سم أو أقل
  {
    digitalWrite(buzzer, HIGH);                   // تشغيل الجرس لإعطاء إنذار بوجود جسم قريب
    delay(500);                                   // إبقاء الجرس يعمل لمدة نصف ثانية
    digitalWrite(buzzer, LOW);                    // إيقاف الجرس بعد انتهاء مدة الإنذار
  }
  else                                            // في حال كانت المسافة أكبر من 30 سم
  {
    digitalWrite(buzzer, LOW);                    // إيقاف الجرس
    delay(1000);                                  // انتظار ثانية كاملة قبل إجراء القياس التالي
  }
}
```

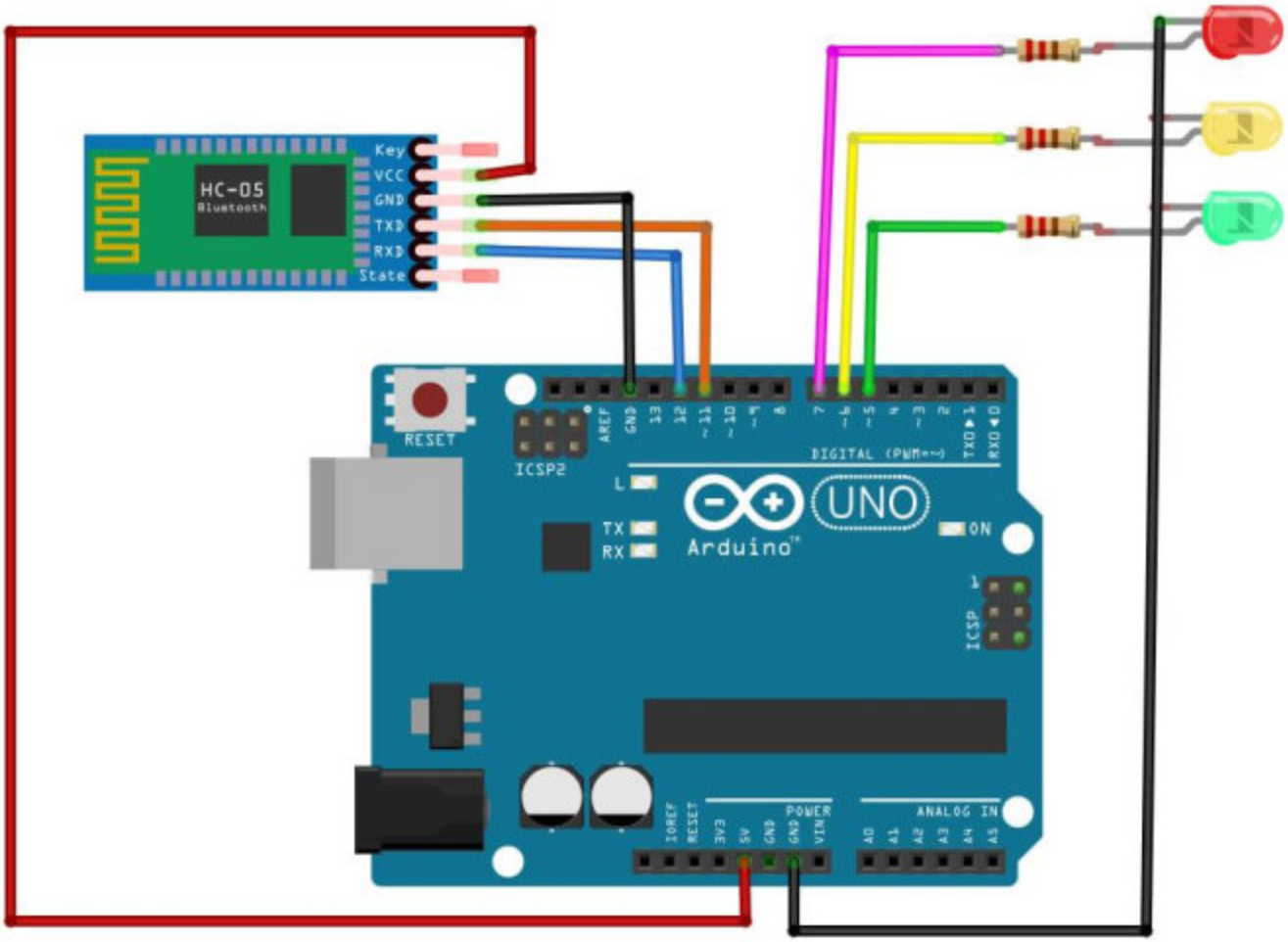
حساس المسافة :



يمكنك مشاهدة فيديو محاكاة لعمل دارة حساس المسافة من هنا

أو عبر مسح الـ QR جانباً :

التطبيق السادس : التحكم عن طريق *Bluetooth*



في هذا التطبيق يتم التحكم بإضاءة ثلاثة ثنائيات ضوئية عن طريق البلوتوث باستخدام وحدة HC_05 التي تستقبل إشارات التحكم الصادرة من هاتف ذكي وذلك من خلال تطبيق بواجهة تخاطب تحوي ثلاث أزرار:

- الزر الأول يرسل المحرف '1' إلى المتحكم.
- الزر الثاني يرسل المحرف '2' إلى المتحكم.
- الزر الثالث يرسل المحرف '3' إلى المتحكم.

عندما يستقبل المتحكم أحد هذه المحارف يضيء الLED الموافق للمحرف لمدة 3 ثوانٍ ثم ينطفئ ... يمكن توسعة عمل هذا التطبيق للتحكم بالأجهزة الكهربائية المختلفة عن طريق الريليه (Relays) .

```
#include <SoftwareSerial.h>          // تضمين مكتبة الاتصال التسلسلي البرمجي لإنشاء منفذ Serial إضافي

int TxD = 11;                        // رجل الإرسال TX الخاصة بوحدة البلوتوث
int RxD = 12;                        // رجل الاستقبال RX الخاصة بوحدة البلوتوث
int num = 0;                         // متغير لتخزين الرقم المستلم من البلوتوث

int redPin = 7;                      // رجل LED الأحمر
int yellowPin = 6;                   // رجل LED الأصفر
int greenPin = 5;                    // رجل LED الأخضر

SoftwareSerial bluetooth(TxD, RxD);  // إنشاء منفذ تسلسلي باسم bluetooth باستخدام الأرجل 11 و12

void setup( )
{
 Serial.begin(9600);                 // تهيئة الاتصال التسلسلي مع الحاسوب بسرعة 9600 بت/ثانية
 bluetooth.begin(9600);              // تهيئة الاتصال مع وحدة البلوتوث بسرعة 9600 بت/ثانية

 pinMode(redPin, OUTPUT);            // ضبط رجل LED الأحمر كمخرج
 pinMode(yellowPin, OUTPUT);         // ضبط رجل LED الأصفر كمخرج
 pinMode(greenPin, OUTPUT);          // ضبط رجل LED الأخضر كمخرج
}

void loop( )
{
 if (bluetooth.available())          // التحقق من وجود بيانات واردة من البلوتوث
 {
   String value = bluetooth.readString();   // قراءة النص المرسل كاملاً من البلوتوث وتخزينه في متغير نصي اسمه value

   num = value.toInt();              // تحويل النص المستلم إلى عدد صحيح وتخزينه في المتغير num

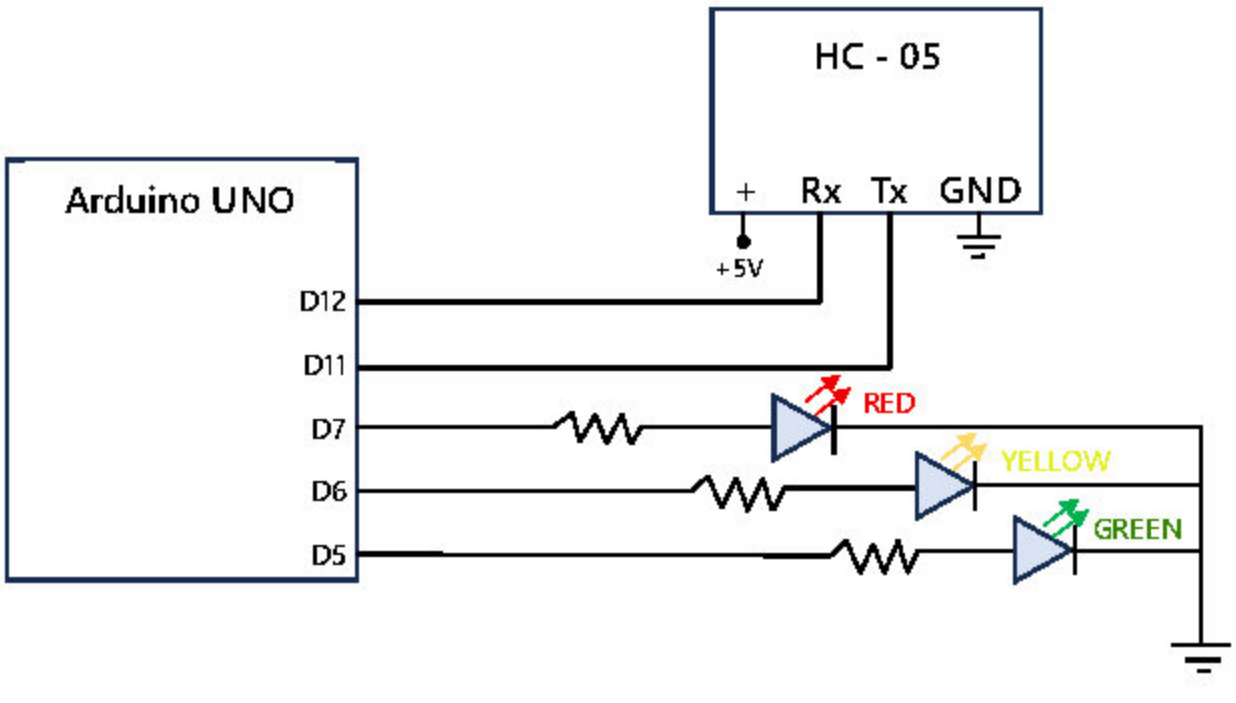
  if (num == 1)                      // إذا كانت القيمة المستلمة تساوي 1
  {
    Serial.println(num);             // عرض القيمة المستلمة على Serial Monitor
    digitalWrite(redPin, HIGH);      // تشغيل LED الأحمر
    delay(3000);                     // الانتظار لمدة 3 ثوانٍ
    digitalWrite(redPin, LOW);       // إطفاء LED الأحمر
  }

  if (num == 2)                      // إذا كانت القيمة المستلمة تساوي 2
  {
     Serial.println(num);            // عرض القيمة المستلمة على Serial Monitor
```

```
    digitalWrite(yellowPin, HIGH);          // تشغيل LED الأصفر
    delay(3000);                            // الانتظار لمدة 3 ثوانٍ
    digitalWrite(yellowPin, LOW);           // إطفاء LED الأصفر
  }

  if (num == 3)                             // إذا كانت القيمة المستلمة تساوي 3
  {
    Serial.println(num);                    // عرض القيمة المستلمة على Serial Monitor
    digitalWrite(greenPin, HIGH             // تشغيل LED الأخضر
    delay(3000)                             // الانتظار لمدة 3 ثوانٍ
    digitalWrite(greenPin, LOW);            // إطفاء LED الأخضر
  }
 }
}
```

البلوتوث:



تطبيق إضافي : التحكم بالأحمال الكبيرة (مطلوب لكن احتمال وروده في الامتحان ضعيف جداً)

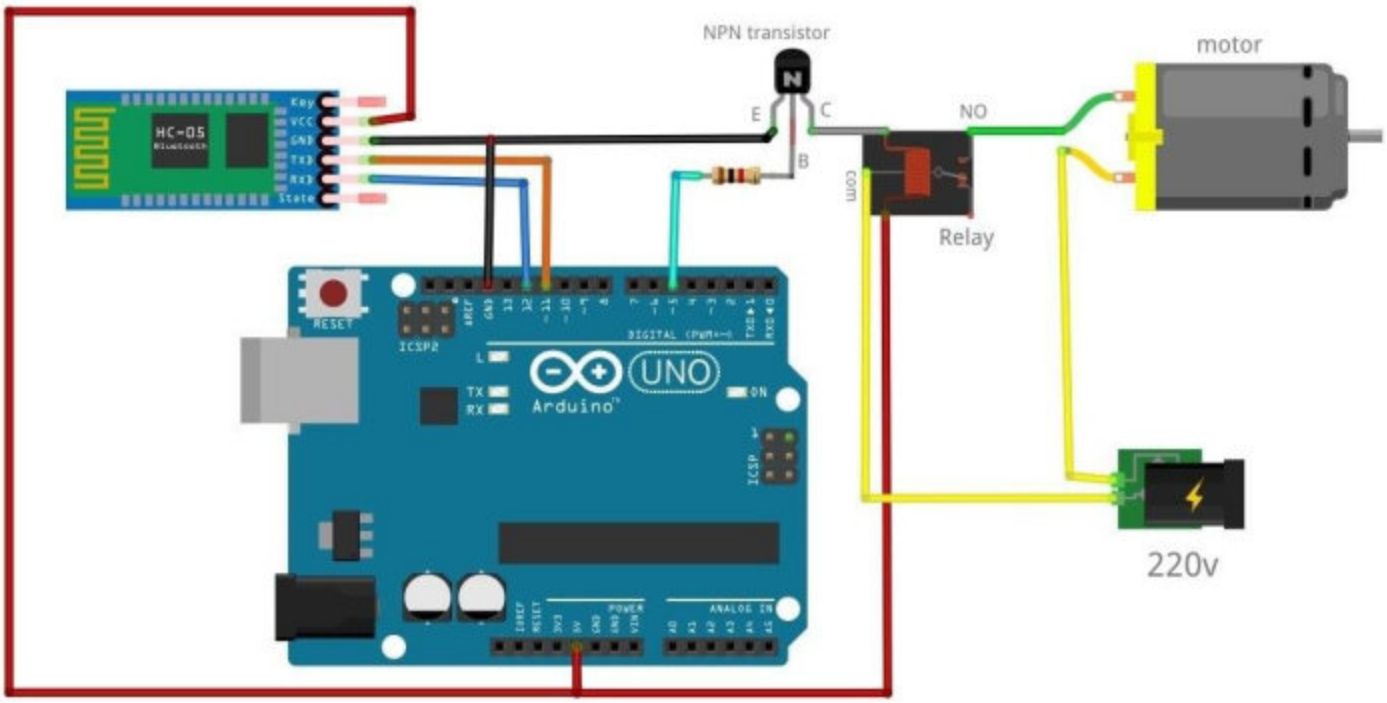
في التجارب التعليمية يتم استخدام LED لتمثيل الحمل المراد التحكم به ، ولكن في التطبيقات العملية يمكن استبداله بريليه للتحكم بأحمال ذات جهد أو تيار كبير مثل المصابيح المنزلية ، المراوح ، المضخات والمحركات .

ونظراً لأن ملف الريليه يحتاج إلى تيار أكبر مما يستطيع خرج الأردوينو توفيره ، يتم استخدام ترانزستور بين الأردوينو و ملف الريليه ليعمل كمفتاح إلكتروني يقوم بتكبير قدرة إشارة التحكم وتشغيل الملف بأمان دون تحميل خرج المتحكم تياراً زائداً قد يؤدي إلى تلفه ، عند وصول إشارة التحكم من الأردوينو إلى الترانزستور ، يسمح بمرور التيار عبر ملف الريليه فيتولد مجال مغناطيسي يؤدي إلى تغيير حالة نقاط التلامس (Contacts) وفتح أو إغلاق دارة الحمل . وتكمن أهمية الريليه في توفير عزل كهربائي بين دائرة التحكم منخفضة الجهد ودائرة القدرة العالية ، مما يسمح بالتحكم بأجهزة كهربائية ذات جهود وتيارات مرتفعة بأمان . وبذلك يمكن استخدام تطبيق البلوتوث نفسه للتحكم عن بعد بأجهزة كهربائية حقيقية بدلاً من الاكتفاء بتشغيل وإطفاء LED فقط.

يمكنك مشاهدة فيديو يشرح آلية عمل الريليه : من هنا

أو عبر مسح الـ QR جانباً :

ملاحظة : قام الدكتور بشرح أفكار الريليه و الترانزستور و كيفية توصيلها لكن المثال التالي غير وارد في المحاضرات وإنما هو مثال خارجي إضافي للتوضيح والمساعدة على الفهم .



في هذا التطبيق سنتحكم بتشغيل و إيقاف محرك يعمل بجهد 220 v عن طريق البلوتوث عند إرسال المحرف '1' من تطبيق الهاتف إلى المتحكم يقوم المتحكم بتشغيل الريليه فتغلق تماساتها ويعمل المحرك ، وعند إرسال المحرف '2' يقوم المتحكم بإيقاف الريليه بالتالي تُفتح تماسات الريليه ويتوقف المحرك عن العمل .

```
#include <SoftwareSerial.h>          // تضمين مكتبة الاتصال التسلسلي البرمجي لإنشاء منفذ Serial إضافي

int TxD = 11;                        // رجل الإرسال TX الخاصة بوحدة البلوتوث
int RxD = 12;                        // رجل الاستقبال RX الخاصة بوحدة البلوتوث
int num = 0;                         // متغير لتخزين الرقم المستلم من البلوتوث

int Relay_Pin = 5;                   // الرجل المربوطة بالريليه

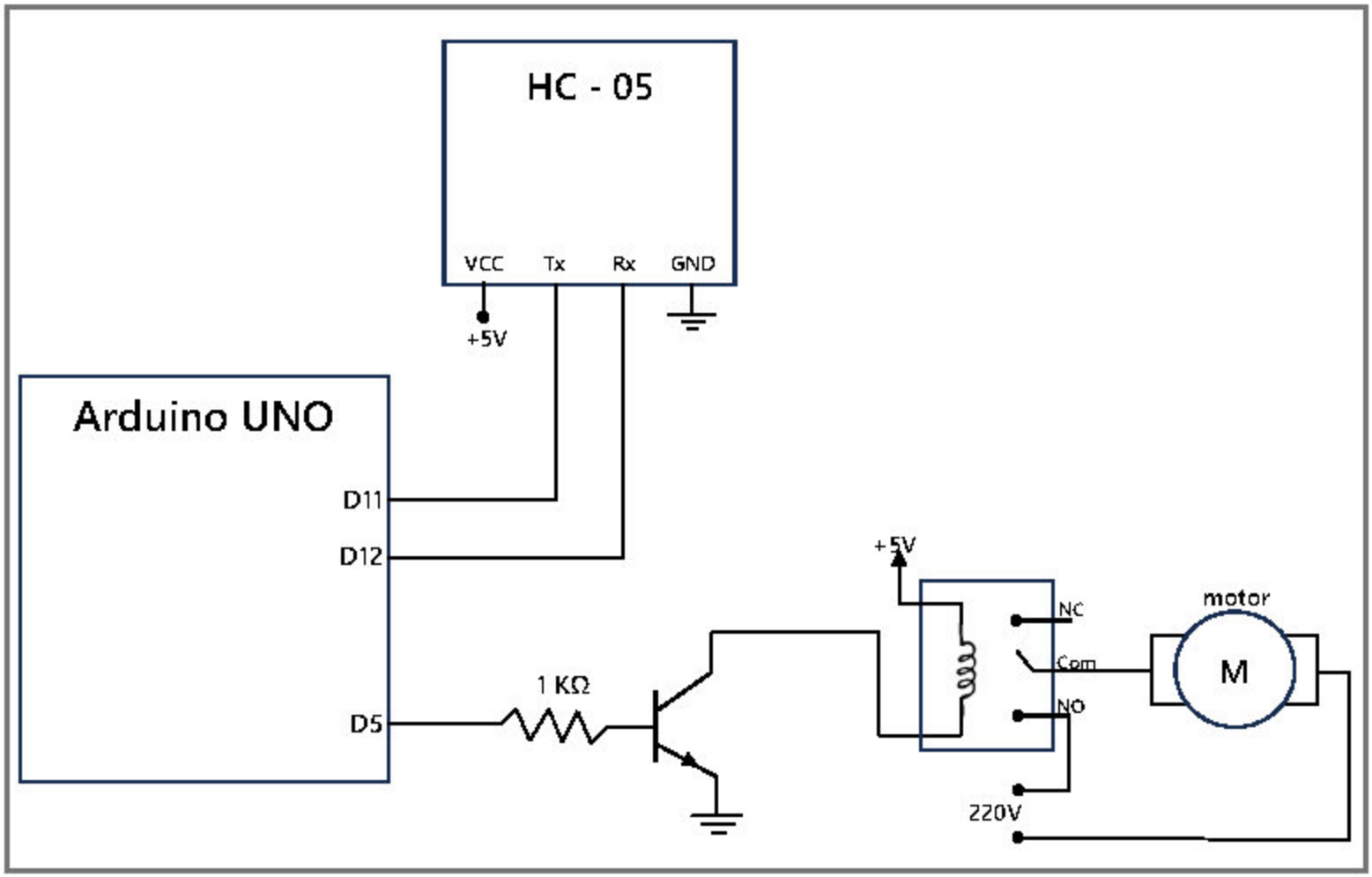
SoftwareSerial bluetooth(TxD, RxD);  // إنشاء منفذ تسلسلي باسم bluetooth باستخدام الارجل 11 و12

void setup()
{
 Serial.begin(9600);                 // تهيئة الاتصال التسلسلي مع الحاسوب بسرعة 9600 بت/ثانية
 bluetooth.begin(9600);              // تهيئة الاتصال مع وحدة البلوتوث بسرعة 9600 بت/ثانية

 pinMode(Relay_Pin, OUTPUT);         // ضبط رجل الريليه كمخرج
}

void loop()
{
 if (bluetooth.available())          // التحقق من وجود بيانات واردة من البلوتوث
 {
   String value = bluetooth.readString();   // قراءة النص المرسل كاملاً من البلوتوث وتخزينه في متغير نصي اسمه value
   num = value.toInt();                     // تحويل النص المستلم إلى عدد صحيح وتخزينه في المتغير num
   if (num == 1)                            // إذا كانت القيمة المستلمة تساوي 1
   {
     Serial.println(num);                   // عرض القيمة المستلمة على Serial Monitor
     digitalWrite(Relay_Pin, HIGH);         // تشغيل الريليه
   }

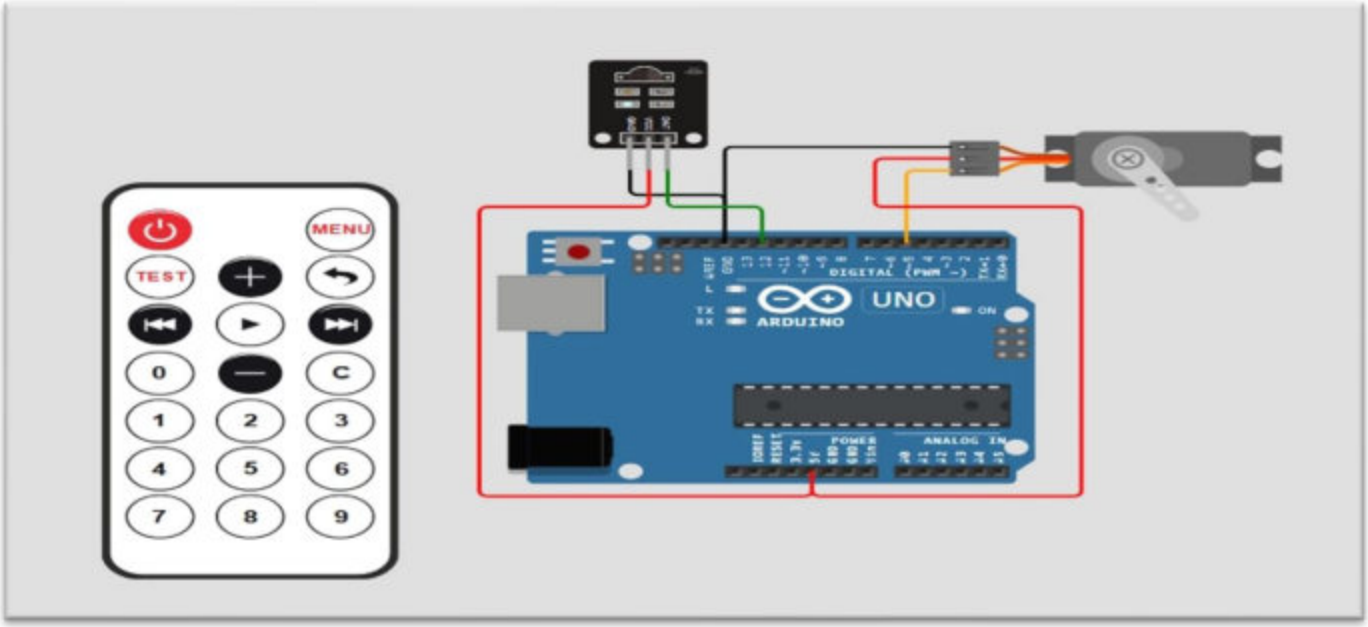
   if (num == 2)                            // إذا كانت القيمة المستلمة تساوي 2
   {
     Serial.println(num);                   // عرض القيمة المستلمة على Serial Monitor
     digitalWrite(Relay_Pin, LOW);          // ايقاف الريليه
   }
 }
}
```


HC - 05
VCC
Tx
Rx
GND
+5V
Arduino UNO
D11
D12
D5
1 KΩ
+5V
NC
Com
NO
motor
M
220V


هانت

التطبيق السابع : التحكم بحركة السيرفو

في هذا التطبيق يطلب التحكم بحركة محرك سيرفو عن طريق جهاز يتحكم بالأشعة تحت الحمراء ، حيث يقوم المحرك بالدوران من زاوية 15 الى الزاوية 165 ذهاباً و ايابأً ، وعند الضغط على زر الإيقاف/الاستئناف في الريموت يقف في مكانه لحين الضغط على الزر مرة أخرى تكون الشيفرة التي يرسلها هذا الزر بحسب جهاز التحكم المستخدم وعادةً ما يكون الترميز المستخدم هو السداسي عشر .




```
#include <Servo.h>                    // تضمين مكتبة التحكم بمحرك ال Servo
#include <IRremote.h>                 // تضمين مكتبة مستقبل الأشعة تحت الحراء IR

Servo myservo;                        // إنشاء كائن اسمه للتحكم بمحرك ال servo

int pos = 15;                         // تخزين الوضع الحالي للمحرك ويبدأ من 15 درجة
int position1 = 15;                   // تحديد الحد الأدنى لحركة المحرك
int position2 = 165;                  // تحديد الحد الأعلى لحركة المحرك
int del = 15;                         // زمن التأخير بين كل حركة وأخرى بالميلي ثانية
int mov_step = 1;                     // مقدار التغير في زاوية المحرك بكل خطوة
bool mov = 1;                         // متغير منطقي لتحديد حالة الحركة (1-حركة ، 0-توقف)
int RECV_PIN = 12;                    // تحديد الرجل المتصلة بمخرج مستقبل IR
IRrecv irrecv(RECV_PIN);              // إنشاء كائن لمستقبل الأشعة تحت الحراء على الرجل 12
decode_results results;               // متغير لتخزين البيانات المستقبلة من الريموت

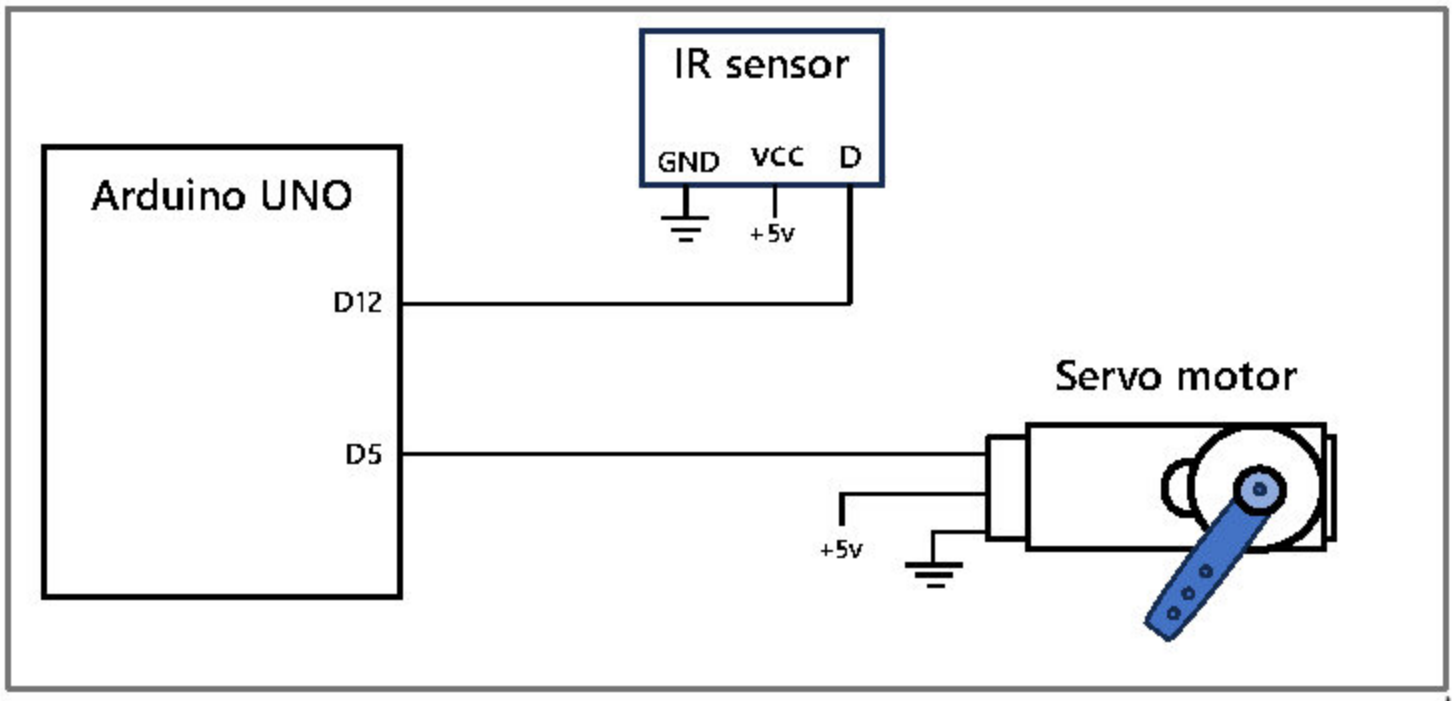
void setup()
```

يمكنك مشاهدة محاكاة للدارة من هنا

او عبر مسح هذا الQR

```
{
 myservo.attach(5);                                  // ربط المحرك بالرجل الرقمية رقم 5
 irrecv.enableIRIn();                                // تشغيل واستقبال إشارات الأشعة تحت الحمراء
 Serial.begin(9600);                                 // تهيئة الاتصال التسلسلي بسرعة 9600 بت/ثانية
}
void loop()

{
 if (mov == 1) {                                     // التحقق فيما إذا كانت الحركة مفعلة
   myservo.write(pos);                               // إرسال الزاوية الحالية إلى محرك ال servo
   delay(del);                                       // الانتظار قليلاً حتى يصل المحرك إلى الزاوية المطلوبة
   pos = pos + mov_step;                             // تحديث الزاوية بإضافة قيمة خطوة الحركة
 if (pos <= position1 || pos >= position2) {         // التحقق من وصول المحرك إلى أحد الحدين
   mov_step = -mov_step;                             // عكس اتجاه الحركة عند الوصول إلى النهاية
 }
}
 if (irrecv.decode(&results)) {                      // التحقق من وصول إشارة جديدة من الريموت
   Serial.println(results.value, HEX);               // طباعة الكود المستلم بالنظام الست عشري على السيريال مونيتور
 if (results.value == 0x5DA2FF00)                    // مقارنة الكود المستلم مع كود الزر في الريموت
 {
 mov = !mov;                                         // تبديل حالة الحركة بين تشغيل وإيقاف
 }
}
 irrecv.resume();                                    // إعادة تفعيل المستقبل لاستقبال الإشارة التالية
 delay(250);                                         // تأخير قصير لمنع تكرار القراءة بسرعة كبيرة
}
```



شرح لبعض تعليمات الكود السابق:

أولاً: تعليمات مكتبة Servo:



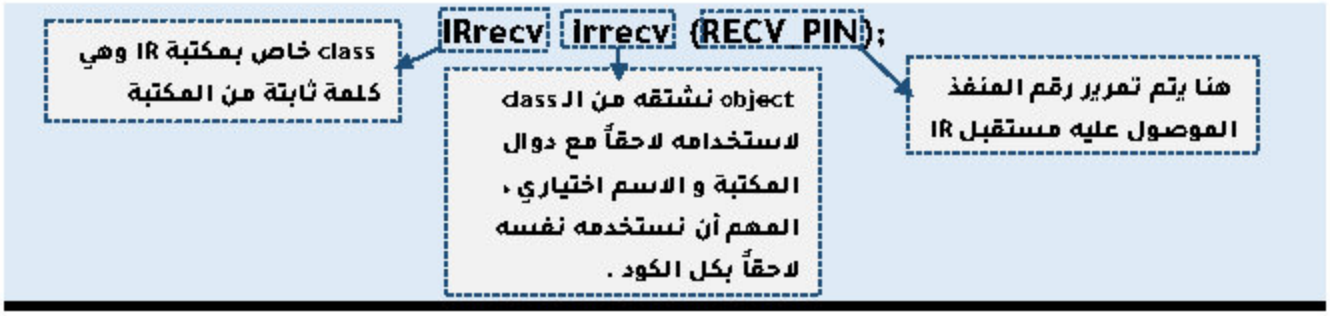
Attach

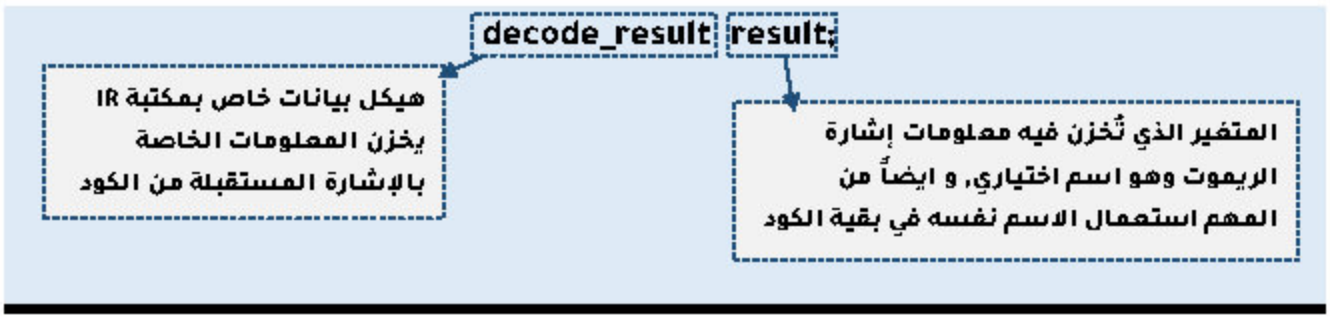
هي دالة خاصة بالمكتبة لتحديد رقم منفذ الموصول عليه المحرك, و يتطلب ان يكون هذا المنفذ يدعم خاصية ال PWM.

Write

على سبيل المثال عند كتابة myservo.Write(0) فإننا نحدد الزاوية المراد للمحرك الدوران بها, أي ان Write مخصصة لإرسال الزاوية.

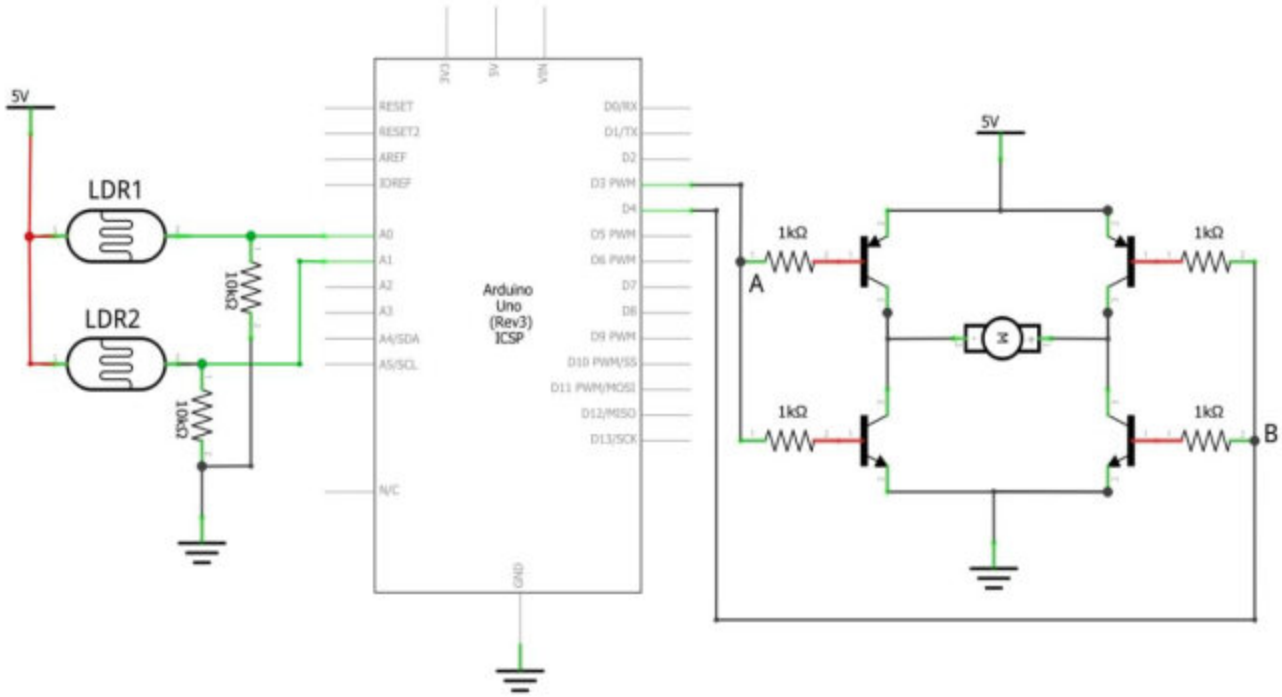
ثانياً: تعليمات مكتبة IR :





التطبيق الثامن : تتبع الشمس

يقوم الأردوينو بمقارنة الإضاءة عند حساسي اضاءة (LDR) و إعطاء أوامر الحركة يميناً او يساراً (A OR B) حيث نهدف لتتبع الخلايا الشمسية لضوء الشمس للحصول على اعلى مردود ممكن لألواح الطاقة الشمسية .

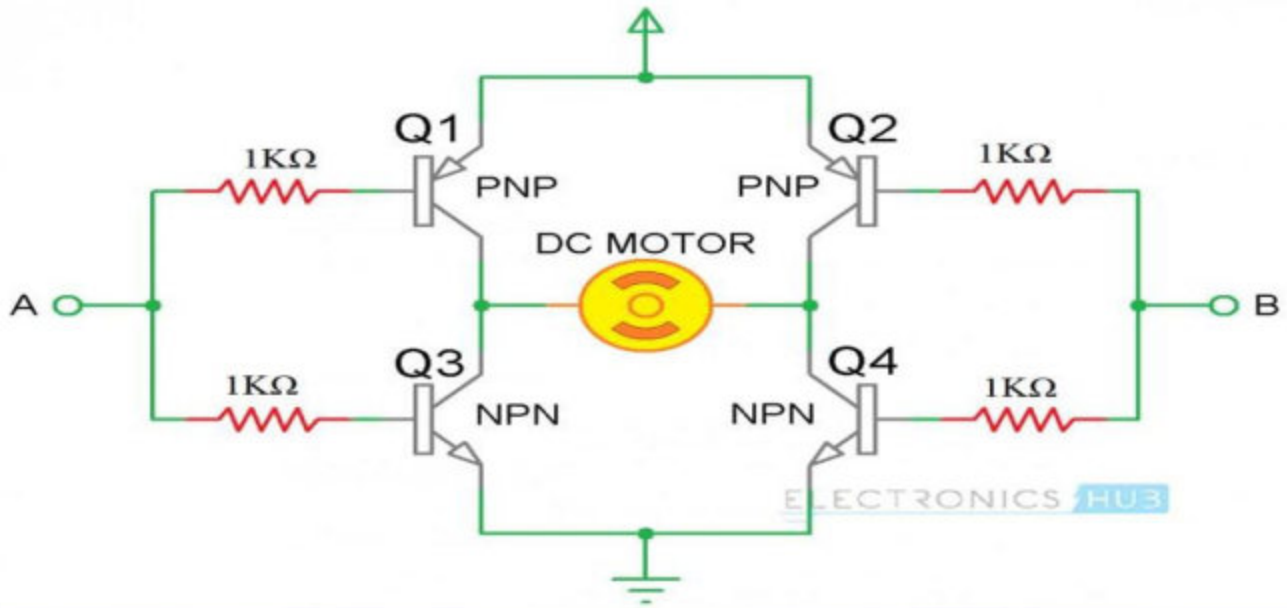



```
int input1 = A0;                    // تعريف الدخل A0 المتصل بالحساس الضوئي الأول (LDR1)
int input2 = A1;                    // تعريف الدخل A1 المتصل بالحساس الضوئي الثاني (LDR2)
int sensorvalue1 = 0;               // متغير لتخزين قراءة الحساس الأول
int sensorvalue2 = 0;               // متغير لتخزين قراءة الحساس الثاني
int Output_A = 3;                   // المخرج الأول المتصل بدارة قيادة المحرك
int Output_B = 4;                   // المخرج الثاني المتصل بدارة قيادة المحرك
int Difference = 0;                 // متغير لتخزين الفرق بين القراءتين
void setup()                        // دالة التهيئة التي تعمل مرة واحدة عند تشغيل الأردوينو
{
  pinMode(Output_A, OUTPUT);        // ضبط الرجل 3 كمخرج
  pinMode(Output_B, OUTPUT);        // ضبط الرجل 4 كمخرج
}
```

```
void loop()                    // الدالة الرئيسية التي تتكرر باستمرار //
{
 sensorvalue1 = analogRead(input1);          // قراءة قيمة الحساس الأول من A0 //
 sensorvalue2 = analogRead(input2);          // قراءة قيمة الحساس الثاني من A1 //
 Difference = abs(sensorvalue1 - sensorvalue2);   // حساب الفرق المطلق بين القراءتين //
 delay(500);                                 // انتظار 500 ميلي ثانية (نصف ثانية) //
 if (Difference > 100)                       // إذا كان الفرق بين القراءتين أكبر من 100 //
 {
  if (sensorvalue2 < sensorvalue1)           // إذا كانت قراءة الحساس الأول أكبر من قراءة الحساس الثاني //
  {
    digitalWrite(Output_A, HIGH);            // جعل المخرج A في الحالة HIGH //
    digitalWrite(Output_B, LOW)              // جعل المخرج B في الحالة LOW //
          /* قيدور المحرك بجهة معينة */
  }
 else
  {
    digitalWrite(Output_A, LOW);             // جعل المخرج A في الحالة LOW //
    digitalWrite(Output_B, HIGH);            // جعل المخرج B في الحالة HIGH //
          /* قيدور المحرك بالجهة الاخرى */
  }
 }
}
```

دارة H-Bridge لقيادة محرك التيار المستمر:

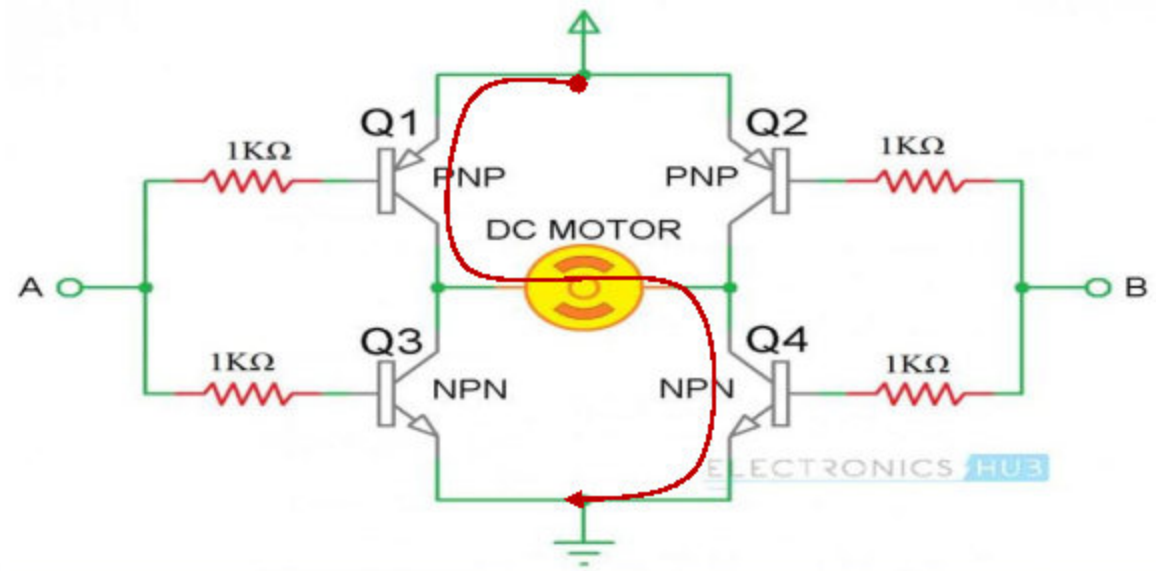
دارة H-Bridge هي دارة الكترونية تستخدم للتحكم باتجاه دوران المحرك المستمر، حيث انها تسمح بتغيير اتجاه مرور التيار داخل المحرك و بالتالي تغيير اتجاه دورانه.



هنا نميز حالتين:

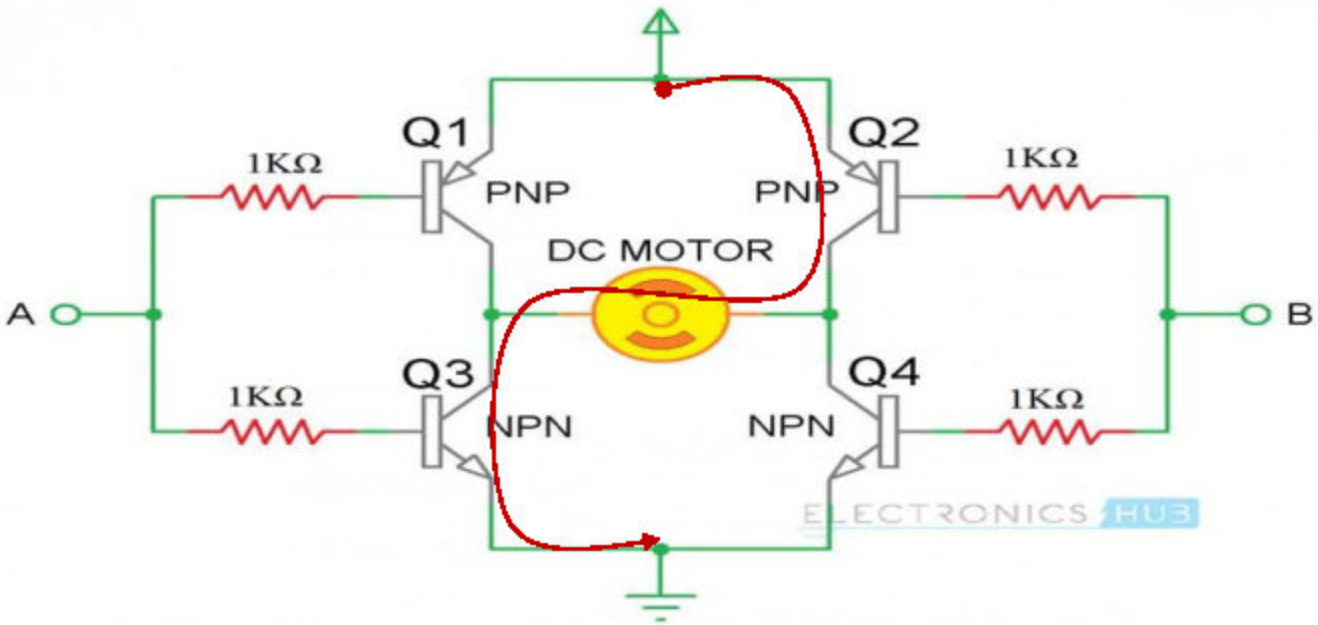
الحالة الأولى $A=0v$ و $B=5v$:

حيث سيعمل كل من الترانزيستورين Q1 و Q4 فيمر التيار بالمحرك بالشكل التالي حسب اتجاه السهم:



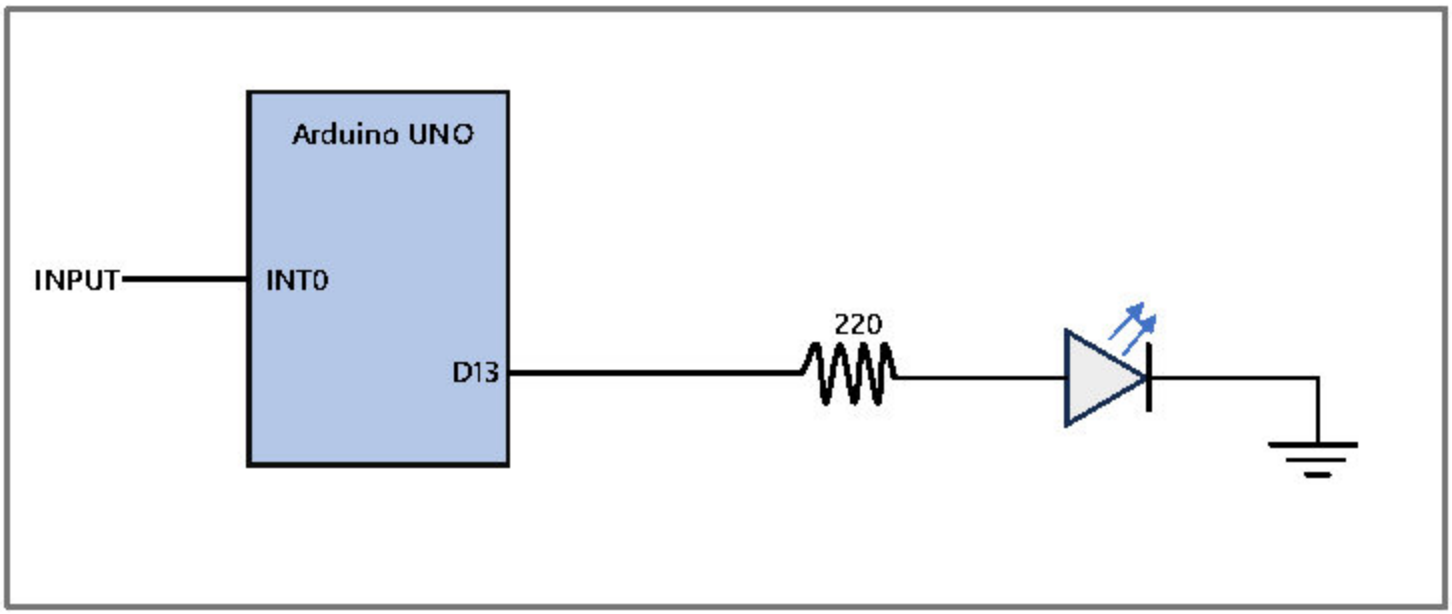
الحالة الثانية: $B=0v$ و $A=5v$:

حيث سيعمل كل من الترانزيستورين Q2 و Q3 فيمر التيار بالمحرك بالشكل التالي حسب اتجاه السهم:



التطبيق التاسع : المقاطعة الخارجية في الـ Arduino

تعاملنا سابقاً مع المقاطعات الخارجية في متحكم Atmega328 وكان التعامل معه معقداً نوعاً ما. و على العكس هنا سنجد التعامل مع الأردوينو أبسط بكثير، التطبيق التالي يشرح تغيير حالة الـ LED الموصول الى الرجل 13 عند كل تغيير أي (any change) على مدخل المقاطعة INT0 في الأردوينو .




```
#define ledPin 13                       // تعريف ثابت باسم ledPin يمثل الرجل الرقمية 13 المتصل بها الـ LED

volatile int status = LOW;              // متغير مشترك بين البرنامج الرئيسي و ISR يبدأ بقيمة LOW (إطفاء LED)
void led_invert_int()                   // دالة خدمة المقاطعة ISR التي تُستدعى عند حدوث المقاطعة الخارجية
{
status = !status;                       // عكس حالة المتغير
}

void setup()
{
pinMode(ledPin, OUTPUT);                // ضبط الرجل 13 كمخرج للتحكم بالـ LED

attachInterrupt(0, led_invert_int, CHANGE);   // attachInterrupt(INT0/INT1, اسم دالة ISR, متى تُستدعى دالة ISR);
}

void loop()
{
digitalWrite(ledPin, status);           // إرسال قيمة المتغير status إلى الرجل 13 لتشغيل أو إطفاء LED
}
```

ملاحظات على الكود السابق:

1. دالة attach Interrupt : هي دالة ثابتة ولها الباراميترات التالية

attachInterrupt(0, Led_invert_int, CHANGE);

- 0: عند قيمة 0 يكون INT0 وعند قيمة 1 يكون INT1
- Led_invert_int: هي الدالة التي نريد استدعائها عند حدوث مقاطعة
- CHANGE: هنا نميز الحالات :
  - CHANGE : عند أي تغيير كان .
  - LOW : عند المستوى المنخفض.
  - FALLING : عند الحافة الهابطة .
  - RISING : عند الحافة الصاعدة .

2. دالة المقاطعة

نقوم بإنشائها بالاسم الذي نريد (في مثالنا كان الاسم led_invert_int) وهو اسم اختياري، فقط المهم هو استدعاؤه نفسه داخل دالة attachInterrup .

3. نلاحظ عند تعريف المتغير status قمنا بكتابة :

Volatile int status = LOW ;

حيث نستخدم كلمة volatile إذا كانت قيمة المتغير يمكن أن تتغير خارج البرنامج الرئيسي، وهنا نلاحظ أن المتغير status تتغير قيمته داخل برنامج خدمة المقاطعة لذلك نستخدم كلمة volatile.



التطبيق العاشر: التخزين في ذاكرة EEPROM

استخدام فكرة هذا التطبيق لتخزين قراءات حساسات مختلفة كدرجة الحرارة والرطوبة و الإضاءة أو أي رقم عشوائي ، يتم ذلك خلال فواصل زمنية معينة ومن ثم استرجاع قيمها وعرضها كما في التطبيق ادناه.

```
#include <EEPROM.h>                 // تضمين مكتبة EEPROM

int addr = 0;                       // عنوان الخلية الحالية في EEPROM

void setup() {

}

void loop() {                       // دالة التكرار الرئيسية
  int val = analogRead(A0) / 4;     // قراءة A0 وتحويلها إلى مجال 0-255 المناسب لـ EEPROM

// int val = byte(random(1000));    // مثال بديل لتوليد قيمة عشوائية وتخزينها

EEPROM.write(addr, val);            // كتابة القيمة val في عنوان addr داخل الذاكرة EEPROM
  addr = addr + 1;                  // الانتقال إلى عنوان الذاكرة التالي

 if (addr == EEPROM.length()) {     // التحقق من الوصول إلى آخر خلية في EEPROM
    addr = 0;                       // العودة إلى أول خلية عند امتلاء الذاكرة
}
  delay(1000);                      // انتظار ثانية واحدة قبل التكرار التالي
}
```

التطبيق الحادي عشر : القراءة من ذاكرة EEPROM

يقوم هذا البرنامج بعرض جميع محتويات EEPROM خليةً تلو الأخرى على الـSerial monitor ، حيث يُظهر كل سطر عنوان الخلية ثم القيمة المخزنة فيها بالصيغة العشرية وينتقل الى الخلية التالية كل ثانية .

```
#include <EEPROM.h>              // تضمين مكتبة EEPROM
int address = 0;                 // متغير يمثل عنوان الخلية الحالية في EEPROM ويبدأ من العنوان 0
byte value;                      // متغير من نوع byte لتخزين القيمة المقروءة من EEPROM (من 0 إلى 255)
void setup() {
  Serial.begin(9600);            // تهيئة الاتصال التسلسلي بسرعة 9600 بت/ثانية لإرسال البيانات إلى Serial Monitor
}
void loop() {
  value = EEPROM.read(address);          // قراءة بايت واحد من EEPROM عند العنوان المحدد بواسطة المتغير address وتخزينه في value
  Serial.print(address);                 // إرسال عنوان الخلية الحالية إلى Serial Monitor دون الانتقال إلى سطر جديد
  Serial.print("\t");                    // إرسال محرف Tab لإنشاء مسافة منظمة بين العنوان والقيمة المعروضة
  Serial.print(value, DEC);              // إرسال القيمة المقروءة بصيغة عشرية (Decimal) إلى Serial Monitor
  Serial.println();                      // الانتقال إلى سطر جديد بعد طباعة العنوان والقيمة
  address = address + 1;                 // زيادة العنوان بمقدار واحد للانتقال إلى الخلية التالية في EEPROM
  if (address == EEPROM.length()) {      // التحقق مما إذا وصل العنوان إلى عدد خلايا EEPROM الكلي
    address = 0;                         // العودة إلى أول خلية (العنوان 0) عند الوصول إلى نهاية الذاكرة
  }
  delay(1000);                           // انتظار ثانية واحدة قبل قراءة الخلية التالية
}
```

ملاحظة : في الامتحان يكون السؤال على الشكل التالي: رسم دارة و كتابة كود .

حيث سيكون هناك دمج بين الأفكار كقراءة قيم حساس حرارة و تخزين القيم في EEPROM عند حدوث مقاطعة مثلاً. لذلك من المهم جدا فهم الاكواد و التطبيقات جيداً(بلا بصم😆) .



تم بعون الله الانتهاء من بحث الأردوينو ...

اكتب ملاحظاتك:

فريقنا يتمنى لكم دراسة موفقة ❤